DATA VERSE AI (Business Intelligence Platform)

Final Project Report by

Muhammad Muavia (051-37766)

Hasnain Ali (051-37721)

M. Shahan Subhani (051-37655)

In Partial Fulfillment

Of the Requirements for the degree

Bachelor of Science in Computer Science (BSCS)

Iqra University H9 Campus

Islamabad, Pakistan

(2026)

Table of Contents

Chapter 1:

Introduction

1.1 Background and Context

Modern organizations produce data constantly. Sales counters, customer conversations, web platforms, and everyday business software all leave behind records, and the volume keeps growing while the formats keep getting messier. Companies therefore lean on analytics to spot patterns and support their decisions. The difficulty is that converting raw records into something a manager can act on normally demands statistics, machine learning, chart design, and programming in Python or SQL, and very few business people carry that mix of skills.

A conventional analysis project moves through several technical stages: the data is cleaned, explored to understand its shape, visualized, and sometimes fed into predictive models. Each stage consumes time and normally requires a trained analyst or a business intelligence specialist. Non-technical staff end up queuing behind the experts, decisions arrive late, and the organization as a whole reacts more slowly than it should. Recent progress in artificial intelligence, and in Large Language Models (LLMs) in particular, opens a different route: analysis can now be requested in plain language.

Instead of writing code, a person can simply talk to the data system and receive summaries, findings, and charts in return. This project presents DataVerse AI, an agent-based analytics platform built around that idea. A user uploads a file, asks questions in a chat, watches charts appear automatically, runs predictions, and reads explanations written in everyday language.

The interface is implemented in Next.js 15 with React 19, while a FastAPI service handles requests asynchronously on the back end. A pluggable language-model layer, which can connect to OpenAI, Google Gemini, Anthropic Claude, or the DeepAnalyze-8B agentic data-science model, lets users phrase requests naturally and turns finished results into readable narration. it is never allowed to produce a number on its own. All figures are calculated deterministically by Pandas and Scikit-learn. Report charts are drawn as hand-built SVG and Chart.js graphics, and model behavior is explained through SHAP attribution, backed by a deterministic feature-importance fallback and counterfactual what-if analysis. Each insight ships with a verifiable

receipt, and every analysis is sealed with a SHA-256 reproducibility certificate. Together these pieces give people without a technical background a dependable way to work with their own data.

1.2 Problem Statement

Because classic BI tooling assumes an expert operator, people who cannot code are locked out and every question must wait for a specialist. Dashboards that never change, complicated software, and predictive models that hide their reasoning make the situation worse. What is missing is a conversational BI platform through which anyone can question a dataset and receive trustworthy findings in ordinary language.

1.3 Aim and Objectives

1.3.1 Aim

To build an AI-driven conversational Business Intelligence platform through which people without technical training can examine data, obtain findings, and make sense of predictive results purely through natural language.

1.3.2 Objectives

- Design a natural language interface so that users can question their data conversationally.
- Automate Exploratory Data Analysis (EDA) so the platform produces dataset summaries, statistics, correlations, and headline findings on its own.
- Supply data visualizations, including bar charts, trend lines, donut charts, histograms, and heatmaps.
- Add classification and regression models together with explainable AI (SHAP attribution with a deterministic feature-importance fallback and counterfactual analysis) so predictions stay transparent.
- Make every displayed figure checkable through computation receipts plus a SHA-256 reproducibility certificate.
- Deliver a friendly interface covering upload, analysis, visualization, and insight generation with zero coding.

1.4 Scope and Delimitations

1.4.1 Scope

The product concentrates on conversational analytics over retail-mart business data. A user can question a dataset the way they would question a colleague. Structured CSV and Excel files are accepted, and a domain guard inspects each upload, confirming it really is retail or mart sales data and politely turning away anything outside that scope. Once accepted, a dataset is profiled automatically: the application runs a full exploratory analysis and presents statistical summaries and visual findings inside the chat itself, which keeps the experience exploratory and engaging. Predictive modeling is included as well, and every AI output is paired with receipt-backed evidence so users can trust what they read. Beyond the software, the project delivers Figma UI/UX prototypes, full SRS documentation, and detailed EDA reports, forming one coherent analytical package.

1.4.2 Delimitations:

Several enterprise-level concerns were deliberately excluded to keep the build manageable. Streaming or real-time analysis is not supported. The product does include multi-user authentication (Supabase Auth issuing JWTs), separation of each user's data, and API rate limiting, but it is not designed for large-scale distributed hosting, and uploads are capped at 50 MB, which rules out datasets beyond typical academic size. Advanced DevOps automation, cloud autoscaling, and deep-learning training were also left out. These boundaries keep the application focused on its purpose: a prototype-level conversational analytics platform that is simple to run and simple to use.

1.5 Significance of Study

The project matters both academically and practically. It widens access to analytics: someone with no coding ability can explore data directly instead of waiting on a BI specialist, which spreads insight across whole departments. Automated exploration plus a conversational front end compresses hours of analytical work into moments. Because visual findings arrive together with understandable predictions, managers get clarity rather than confusing model output. SHAP-based attribution, narrated in plain language and supported by verifiable receipts, shows the reasoning behind each prediction and builds confidence in the results. On the research side, the

work contributes to natural language interfaces, human-computer interaction, AI-assisted analytics, and interpretable machine learning. On the practical side, it hands small businesses, which rarely afford analytics teams, a serious analytical capability at minimal cost.

1.6 Report Structure

The document contains eight chapters. Chapter 1 sets out the background, the problem, the objectives, the significance, and the boundaries of the work. Chapter 2 reviews prior research and tooling and isolates the research gaps. Chapter 3 states the research questions, hypotheses, evaluation criteria, and concrete goals. Chapter 4 covers the methodology: architecture, workflow design, algorithms, and technology choices. Chapter 5 walks through the proposed solution and the UI/UX design with screens and diagrams. Chapter 6 documents the dataset, profiling, visualizations, and analysis outcomes. Chapter 7 reports results, evaluation, usability observations, and performance. Chapter 8 closes with contributions, future directions, citations, diagrams, and supporting material.

Chapter 2:

Literature Review

2.1 Introduction

[1] BIREC approaches conversational business intelligence as guided data analysis: the user speaks in natural language and the system helps organize and refine each analytical step. The work shows that pairing language interfaces with familiar BI workflows lowers the entry barrier for non-technical people. Even so, BIREC stays within guided exploration; it does not offer an automated end-to-end pipeline that would also cover visualization, predictive modeling, and explainability. [1]

[2] ChatBI is a framework that converts natural language into complex business intelligence SQL. Its focus is closing the distance between what a user means and the structured query that expresses it, so that people can work with databases without knowing SQL. The system raises accessibility and query accuracy, yet its reach ends at NL-to-SQL translation; downstream stages such as exploratory analysis, chart generation, and model-driven insight are not addressed. [2]

[3] This survey maps the intersection of Explainable AI (XAI) and Large Language Models, showing how LLMs can make AI systems easier to interpret. It organizes transparency techniques, discusses the difficulty of aligning generative models with explainability requirements, and points out an open gap: no unified framework yet joins reasoning, explanation, and interactive analytics in one system. [3]

[4] The survey on approaches, limitations, and applications of Explainable AI traces how XAI techniques evolved across domains and where they are used in practice. It documents recurring trade-offs between model accuracy and interpretability and the difficulty of scaling explanations to complex systems, and it argues for explanation methods that adapt to the user and the domain so that trust in AI can grow. [4]

[5] This review examines how Large Language Models are applied to text-to-SQL, analyzing methods, datasets, and evaluation practice. It finds that LLMs have pushed SQL generation accuracy and robustness forward considerably, while noting stubborn problems: complicated queries, generalizing across schemas, and guaranteeing that generated SQL executes correctly in real business settings. [5]

2.2 Natural Language to SQL (NL2SQL)

Many conversational analytics systems are built on NL2SQL: the user types a question in ordinary words and the application emits runnable SQL. The 2024 survey by Shi et al. offers a wide view of LLM-powered Text-to-SQL, covering both progress and open problems. Current models cope with harder queries, including nested subqueries and joins across tables, yet several failure modes persist: ambiguous column references, faulty multi-table joins, broken schema constraints, and lost context across conversation turns. The practical consequence is SQL that parses correctly but means the wrong thing, which is dangerous in BI settings where the numbers must be right.

As a remedy, Shi et al. recommend hybrid pipelines in which the LLM interprets intent and drafts SQL while rule-based or programmatic validators enforce schema consistency and correctness. DataVerse AI follows the same philosophy: a language model (reached through an OpenAI-compatible provider layer) reads the user's intent, and fixed Python modules carry out the actual analysis, which removes the risk of hallucinated or invalid queries. Agentic AI surveys reach matching conclusions. Plaat et al. and Brohi et al. observe that LLM agents perform best when combined with external tools, structured knowledge, and explicit reasoning steps rather than computing everything internally. In short, an LLM alone cannot deliver dependable analytics over structured data. a deterministic backend must execute and validate.

Taken together, the NL2SQL and agentic literature endorses the design decision behind DataVerse AI: let language models interpret and orchestrate, and let deterministic code perform EDA, visualization, and modeling.

2.3 Automated Exploratory Data Analysis (EDA)

Tools such as Pandas Profiling, Sweetviz, and AutoViz emerged to speed up the production of summary statistics, correlation matrices, and dataset visuals. They generate quick reports exposing distributions, missing-data patterns, and possible anomalies, all of which matter before modeling begins. Their weakness is heritage: they were written for code-first workflows, assuming a user who is comfortable with Python scripts and long HTML reports rather than conversation. Work from 2024 and 2025, summarized across LLM-focused surveys, shows growing interest in wiring LLMs into EDA automation so the model can narrate statistical findings. Studies of LLM-driven analysis automation report that models can describe patterns,

outliers, and correlations, and can even propose the next plot or analysis step, which makes exploration far friendlier for non-experts.

Dressel et al. present the XAI for All framework, asking whether LLMs can translate dense explainability output for broader audiences. Although aimed at XAI rather than EDA, the study shows that LLMs can restate technical artifacts, such as feature-importance scores or local explanation vectors, as accessible narratives. DataVerse AI applies the same idea to exploration: numeric summaries and visuals arrive together with plain-language descriptions of distributions, correlations, and data quality. The same body of work carries a warning, though: narrative summaries drift into error when they are not anchored to explicit computation. The safe pattern, and the one DataVerse AI implements, is to compute all statistics and plots with dependable Python libraries such as Pandas and NumPy and restrict the LLM to explaining verified results.

Taken together, the EDA literature indicates that automated tooling and LLM narration exist but have not been properly fused into conversational BI, and that connecting EDA automation to language interfaces and downstream modeling remains open work.

2.4 Explainable Artificial Intelligence (XAI)

For BI platforms that include machine learning, explainability has moved from nice-to-have to requirement, since stakeholders expect predictions they can audit. Yang et al. survey the XAI landscape, spanning global techniques such as feature importance and surrogate models alongside local ones such as LIME and SHAP that explain single predictions [4]. They stress that explainability underpins responsible deployment in finance, healthcare, governance, and other high-stakes fields [4]. The catch is that these methods speak in technical artifacts, such as SHAP plots and perturbation analyses, which mean little to a business user.

Cambria et al. study the pairing of XAI with LLMs and find that language models can serve as explanation generators, recasting numeric or graphical XAI output as readable prose [3]. Their survey, XAI meets LLMs, shows models clarifying which features matter and how each pushes a prediction up or down.

DataVerse AI builds its explainability layer on these findings: SHAP-based global attribution (with a deterministic feature-importance fallback whenever SHAP is unavailable) plus counterfactual what-if analysis handle the interpretation, and the LLM layer rewrites those outputs as conversational narration users can question further [4]. The literature also exposes the

gap this project targets: XAI research rarely reaches conversational BI, where explanations must live inside an ongoing dialogue rather than on a static dashboard [4].

2.5 Visualization Recommendation Systems

Visualization recommenders choose an appropriate chart automatically from the user's intent and the shape of the data, sparing the user manual design decisions. Li et al. (2022) describe an intent-aware recommendation framework that learns to map high-level goals, comparison, trend detection, or distribution analysis, onto fitting visual encodings [visualization papers, 2022]. Understanding intent, they show, yields more relevant and clearer charts. Kavaz et al. (2023) extend the idea to chatbot interfaces where the user describes the desired view in everyday words and receives recommended visualizations [visualization papers, 2023]. Their scoping review positions language-driven visualization as an emerging field with several prototypes that turn text into charts and dashboards.

Yet, as the surveys by Cambria et al. and others note, most NL-to-visualization systems live as standalone tools, disconnected from wider BI workflows that include exploration, modeling, and explainability. They also tend to lack multi-turn ability and lose context when the user refines a chart step by step. Those shortcomings motivate platforms like DataVerse AI, where chart recommendation sits inside a complete conversational pipeline covering EDA, prediction, and XAI.

2.6 Integrated LLM-based Analytics Systems

Recent agentic-AI surveys converge on hybrid architectures as the future of analytics: the LLM coordinates tools, data, and workflow rather than doing every task itself [8], [9]. Plaat et al. characterize agentic LLMs as systems capable of multi-step reasoning, planning, and tool use, able to steer complex analytical processes by invoking external tools for computation, retrieval, or visualization [8]. Their survey walks through agents that fetch data, execute code, and produce reports from high-level instructions [8]. Brohi et al. chart the broader agentic AI landscape, covering analytics, governance, and decision support, and note that agentic designs shine wherever iterative refinement and continuous interaction are needed, which describes BI work exactly [9].

At the same time, Shi et al. and Cambria et al. warn that pipelines made only of LLMs stumble on structured data and rule-bound tasks. Even the strongest Text-to-SQL models mishandle schema grounding and constraints [5], and explanations produced without tool support can mislead [3]. The shared conclusion: let LLMs interpret intent, plan, and communicate, while reliable backend systems execute EDA, visualization, modeling, and XAI [5]. DataVerse AI embodies this principle through a hybrid design in which language models converse and orchestrate while Python libraries, ML frameworks, and XAI tooling do the computation. DeepAnalyze-8B, the first agentic LLM built specifically for autonomous data science, is supported as a reasoning provider in precisely that role.

2.7 Comparative Analysis Table

Table 2.1: Comparative Review of Related BI and Conversational Analytics Systems

Study / Tool / System	Purpose	Strengths	Limitations	Reference
BIREC	Conversational BI exploration	Intent extraction, guided tasks	No EDA, no ML, no visualization	[1]
ChatBI	Natural language to BI SQL	Handles complex SQL joins	No EDA, no explainability	[2]
XAI meets LLMs	Relationship between XAI & LLMs	Narrative explanation generation	Not BI-specific	[3]
Explainable AI Survey	Overview of XAI methods	Covers SHAP, LIME	Not conversational	[4]
LLM Text-to-SQL Survey	NL to SQL via LLMs	State-of-the-art LLM approaches	Still inaccurate on schemas	[5]
Explainable Generative AI	Explainability for Generative/LLM systems	Discusses transparency & trust	No BI integration	[6]
XAI for All	LLM simplification of XAI	User-friendly explanations	Not BI-specific	[7]
Agentic LLM	LLM agents + tool use	Strong reasoning + action	No BI implementation	[8]
Agentic AI Landscape	Applications of Agentic AI	Multistep reasoning	Not BI-focused	[9]

2.7.1 Summary of Comparative Analysis Table:

Reading these systems side by side exposes how fragmented the field is. BIREC is capable at guided conversational exploration, offering intent extraction and task suggestions, but it has no automated EDA, no machine learning, and no advanced visualization. ChatBI translates questions into sophisticated BI SQL and copes well with multi-table joins, yet ships neither

integrated exploration nor explainability. The XAI meets LLMs survey illuminates explanation methods and the role LLMs can play in interpretability, but it is not a BI platform and offers no conversational analytics workflow. The overall picture: individual capabilities, NL-to-SQL, chart recommendation, XAI, have each matured, while no single system unifies them into a complete conversational BI pipeline [3].

2.8 Research Gap Identification

Synthesizing the 2022 to 2025 literature surfaces five principal gaps:

2.8.1 Gap 1: Lack of unified conversational BI systems.

Published work concentrates on single functions: SQL translation (ChatBI), guided exploration (BIREC), or explainability inside general-purpose ML systems [3]. No platform yet joins conversational querying, exploratory analysis, visualization, predictive modeling, and explanation into one continuous workflow.

2.8.2 Gap 2: No conversational EDA automation.

Automated EDA tools and LLM-driven analysis studies prove that descriptive statistics and visual summaries can be generated without human effort, yet access still runs through code or rigid interfaces rather than conversation [3]. Systems that launch a full EDA workflow simply because the user asked a question in plain language remain rare.

2.8.3 Gap 3: No conversational XAI integration.

Frameworks such as SHAP and LIME, along with the XAI and GenXAI surveys, concentrate on producing sound explanations and articulating transparency needs [4], and XAI-LLM studies show language models can deliver those explanations [3]. What is missing is a documented system that embeds XAI inside a conversational BI interface where users can request, refine, and drill into explanations live.

2.8.4 Gap 4: Need for hybrid LLM + deterministic pipelines.

Text-to-SQL and agentic surveys agree that LLM-only stacks are unreliable for structured analytics and need deterministic companions for schema-aware reasoning and safe execution [5]. Many prototypes nevertheless still lean on the LLM for both interpretation and execution.

Concrete implementations are needed that pair LLM intent understanding with solid Python backends for EDA, visualization, ML, and XAI.

2.8.5 Gap 5: Absence of a full NL to EDA to Visualization to Prediction to XAI pipeline.

No reviewed system carries the entire BI lifecycle in conversational form. tools stop after query execution or charting, or address explainability without ingestion, EDA, and modeling [1]. Absent from the literature is one platform where a user uploads data, asks questions in ordinary language, receives EDA summaries and recommended charts, runs predictive models, and reads SHAP-based explanations, all in a single conversation.

DataVerse AI is aimed squarely at these five gaps. It couples LLM-based language understanding with deterministic Python analytics and merges exploration, visualization, prediction, and explainable AI into one conversational business intelligence system.

2.9 Summary of Literature Review

In summary, advances across conversational BI, NL2SQL, visualization recommendation, EDA automation, XAI, and agentic AI supply strong building blocks for interactive analytics [1]. BIREC and ChatBI prove that language interaction lowers the barrier to data, though neither completes the analytical workflow [2]. The NL2SQL surveys show LLMs to be powerful but error-prone on their own, arguing for hybrids that pair interpretation with deterministic execution [5]. EDA and XAI research shows that exploration and model explanation can both be simplified into narratives non-experts follow, particularly with LLM help [3]. Work on visualization recommendation and agentic AI points toward intent-aware charting and tool-using agents, neither yet folded into BI pipelines [3].

Despite this progress, the literature keeps conversational BI, NL2SQL, visualization, EDA, prediction, and XAI in separate silos. A unified conversational platform covering the full NL to EDA to Visualization to Prediction to XAI path does not yet exist [1]. DataVerse AI is proposed to fill exactly that space: one coherent hybrid system that keeps analytics reliable, transparent, and interpretable while opening it to non-technical users.

Chapter 3:

Research Problem and Objectives

3.1 Overview

This chapter pins down the research problem and the goals of DataVerse AI. Building on the gaps identified in the review, it argues for a single conversational Business Intelligence system that lets non-technical users run complete analyses. The intended outcome is an all-in-one platform handling Exploratory Data Analysis (EDA), visualization, predictive modeling, and explainable machine learning through natural conversation.

3.1.1 Research Problem

Data drives more and more organizational decisions, yet the workflows that produce insight remain deeply technical. Business users wait on data scientists and BI professionals for exploration, charts, models, and interpretation, which delays insight. BIREC [1], ChatBI [2], and the recent surveys of conversational and natural language interfaces [8], [9] confirm rising interest in conversational analytics while also cataloging the limits of current systems: conversational BI products mostly generate SQL instead of supporting whole workflows, automated EDA lives apart from chat interfaces, visualization recommendation rarely accepts natural language, XAI methods like SHAP and LIME have not reached conversational BI, and agentic frameworks theorize about reasoning and tool use without practical BI applications. The gap in both research and tooling is unmistakable.

Stated precisely: no unified conversational BI system currently allows a non-technical user to run EDA, produce visual analytics, train predictive models, and receive explainable ML output through natural language alone. The consequences are slow BI cycles, permanent dependence on technical staff, inefficient decisions, and opaque predictions, all of which shrink the value analytics delivers to an organization. DataVerse AI answers this problem by fusing conversational interaction with automated analytics and explanation.

3.1.2 Research Questions

The study pursues research questions covering the interpretability, effectiveness, usability, and dependability of a conversational BI system.

- RQ1: How accurately can a conversational interface read natural language analytical requests and route them to the right operation, whether EDA, visualization, or predictive modeling?
- RQ2: Can automated EDA triggered by plain-language prompts return findings that are both accurate and useful?

3.1.3 Research Objectives

The overarching aim is to build and evaluate an AI-driven conversational BI platform through which non-technical users can explore data, generate charts, run predictive models, and understand ML results using ordinary language. That aim breaks into concrete goals.

First, develop a language-model module that reads analytical questions and binds them to backend functions. Second, construct automated EDA workflows producing descriptive statistics, correlations, outlier detection, and narrative findings. Third, implement predictive models with Scikit-learn, specifically Ridge regression and Random Forest with automatic task selection.

Fourth, integrate SHAP explainability with a deterministic feature-importance fallback and counterfactual analysis so every prediction can be explained clearly. Further goals include a friendly multi-turn conversational UI/UX that keeps interactions smooth across tasks, evaluation of accuracy, usability, and reliability, and complete documentation spanning SRS specifications, UI/UX designs, EDA outputs, and this final report.

3.1.4 Research Scope

The scope is drawn tightly so the work stays feasible while still answering the core problem. The application accepts structured CSV and Excel datasets from the retail-mart domain, with a domain guard that declines out-of-scope uploads and explains why. It offers automated exploration covering descriptive statistics, outlier detection, and correlation analysis. visual output including heatmaps, histograms, bar charts, and trend lines. language understanding through the LLM layer. predictive classification and regression. and explainability through SHAP with a deterministic fallback. A chat interface is the primary mode of use.

Outside the scope sit big-data processing, streaming pipelines, and enterprise-scale infrastructure, along with deep neural network training and real-time production dashboards, none of which belong in an academic prototype of this kind.

3.1.5 Research Methodology Overview

The methodology proceeds in order. Requirements analysis first establishes functional and non-functional specifications. System design follows, covering architecture, analytical workflows, and UI/UX structure. Implementation uses Next.js for the frontend, FastAPI for the backend, and Python modules for EDA, ML, and explainability. Evaluation then measures usability, accuracy, reliability, and clarity of explanation. Documentation completes the cycle: architecture diagrams, SRS specifications, design reports, and evaluation results that support academic rigor and reproducibility.

3.1.6 Success Criteria and Evaluation Metrics

DataVerse AI is judged on technical, usability, and reliability measures. Technical measures cover intent interpretation accuracy, correctness of automated EDA output, relevance and readability of charts, predictive model performance, and comprehensibility of explanations. Usability measures include task completion time, System Usability Scale (SUS) scores, and user satisfaction. Reliability measures examine stability across datasets, suppression of LLM errors, and consistency of results across repeated runs, reinforced by an automated backend suite of 214 tests and by SHA-256 reproducibility certificates attached to analysis output. Together these determine the effectiveness of the application.

3.2 System Architecture Diagram

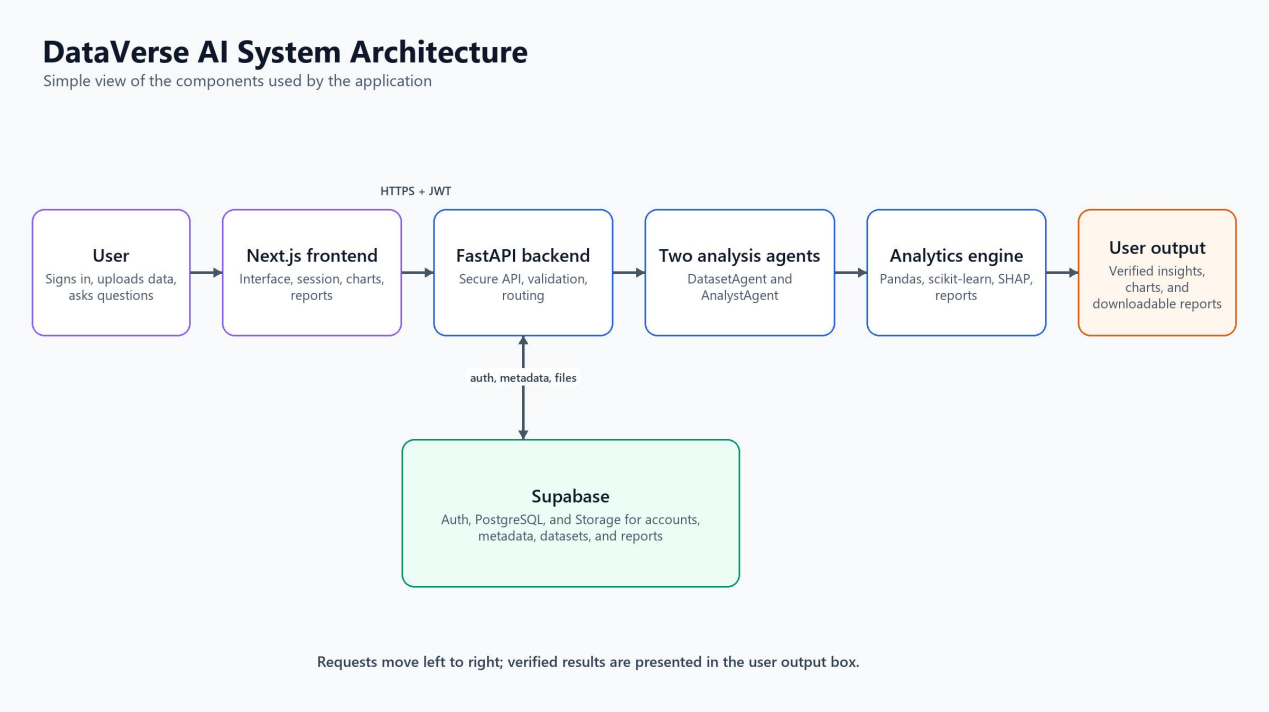

Figure 1: Updated System Architecture and Output Flow

Figure 1 presents the implemented request path. A signed-in user submits a dataset or question through the Next.js interface. The frontend sends the request to FastAPI over HTTPS with the Supabase JWT. FastAPI validates and routes the request to DatasetAgent and AnalystAgent. The analytics engine performs the calculations and produces verified insights, charts, or downloadable reports for the final output box. A separate bidirectional connection allows FastAPI to read and write account state, metadata, datasets, and reports in Supabase.

3.3 Use Case Diagram

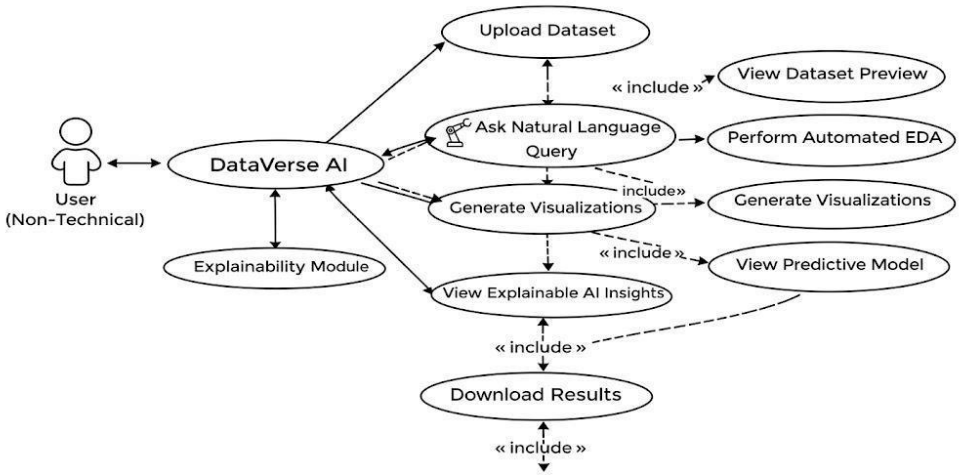

Figure 2: Use Case Diagram

The use case diagram captures how a non-technical person operates DataVerse AI. After uploading a dataset, the user asks questions in ordinary language. the application responds with automatic exploratory analysis, generated charts, and predictive models, plus explainable AI output that makes the results understandable. Dataset preview, output viewing, and downloads round out the interactions, putting advanced analysis within reach of anyone.

3.4 EDA

3.4.1 Missing Values Per Column

Locates columns containing missing entries to judge data quality and plan any cleaning.

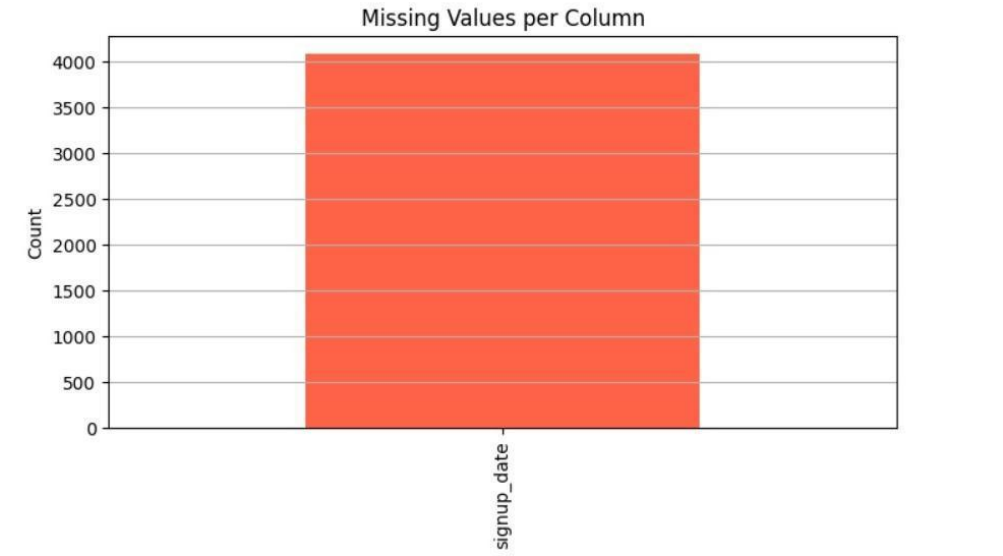

Figure 3: Missing Values Analysis

3.4.2 Product Category Distribution

Counts transactions per product category to reveal customer preferences.

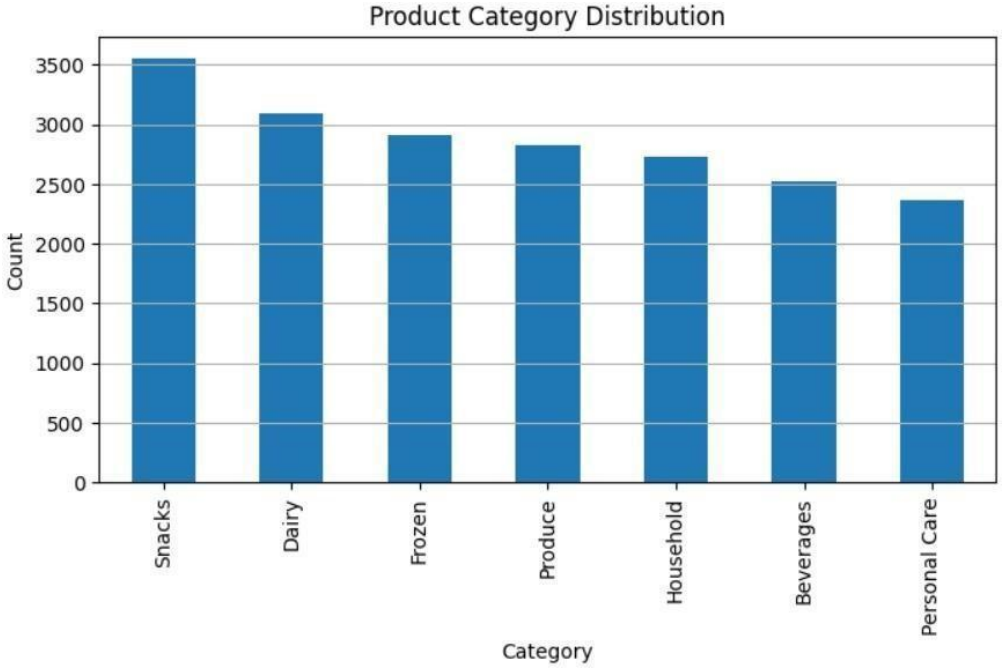

Figure 4: Product Category Distribution

3.4.3 Payment Method Distribution

Breaks down usage across payment methods to identify the dominant transaction modes.

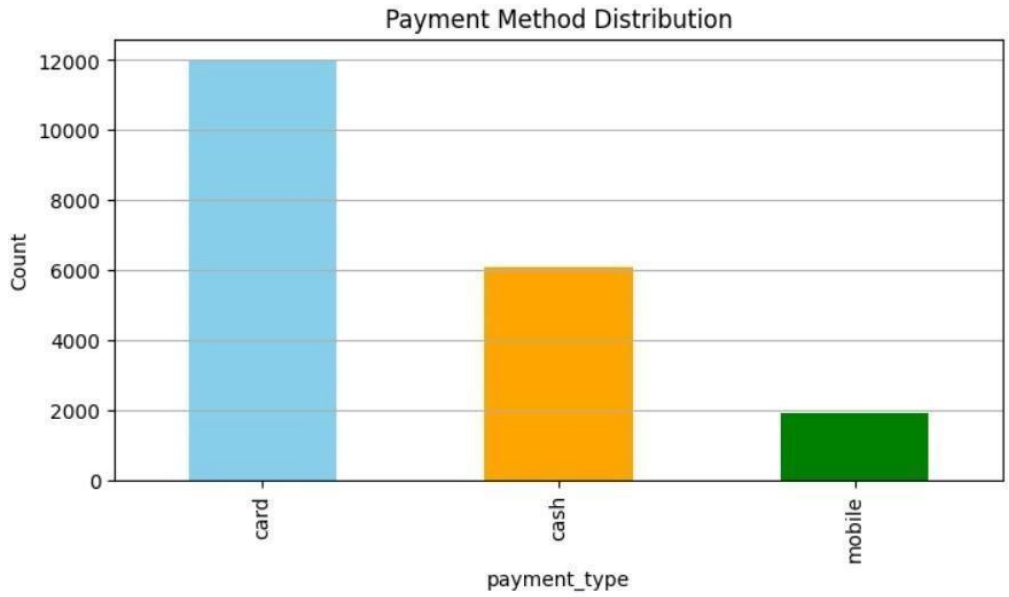

Figure 5: Payment Method Distribution

3.4.4 Tier Distribution

Profiles the customer or product tiers to expose segmentation patterns.

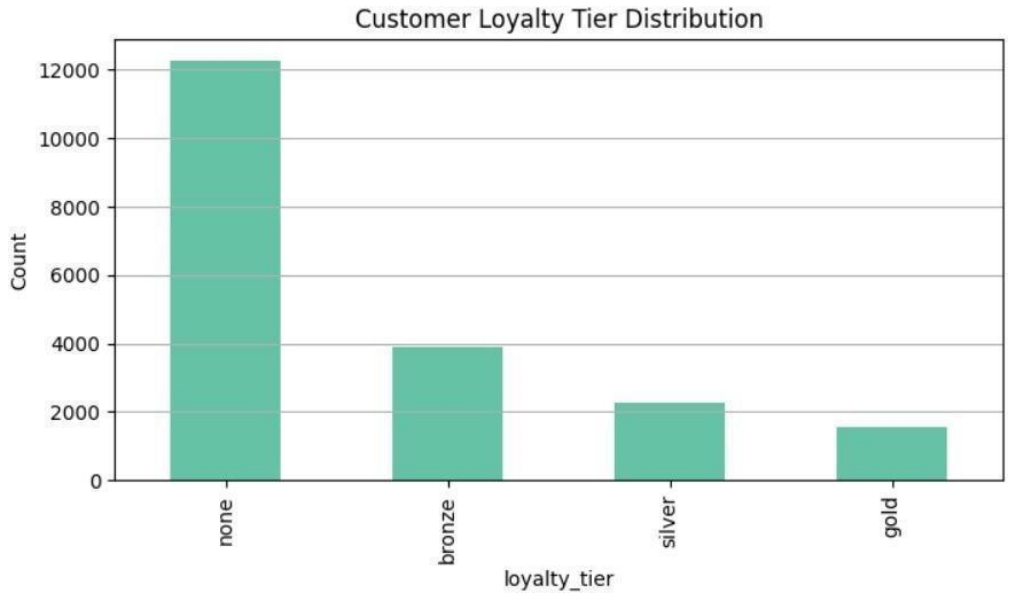

Figure 6: Tier Distribution

3.4.5 Quantity Purchased Distribution

Shows how purchase quantities spread across transactions, highlighting typical buying behavior.

Figure 7: Quantity Purchased Distribution

3.4.6 Unit Price Distribution

Examines the range and spread of unit prices to surface pricing patterns and anomalies.

Figure 8: Unit Price Distribution

3.4.7 Transaction Amount Distribution

Plots the distribution of transaction values to characterize spending behavior.

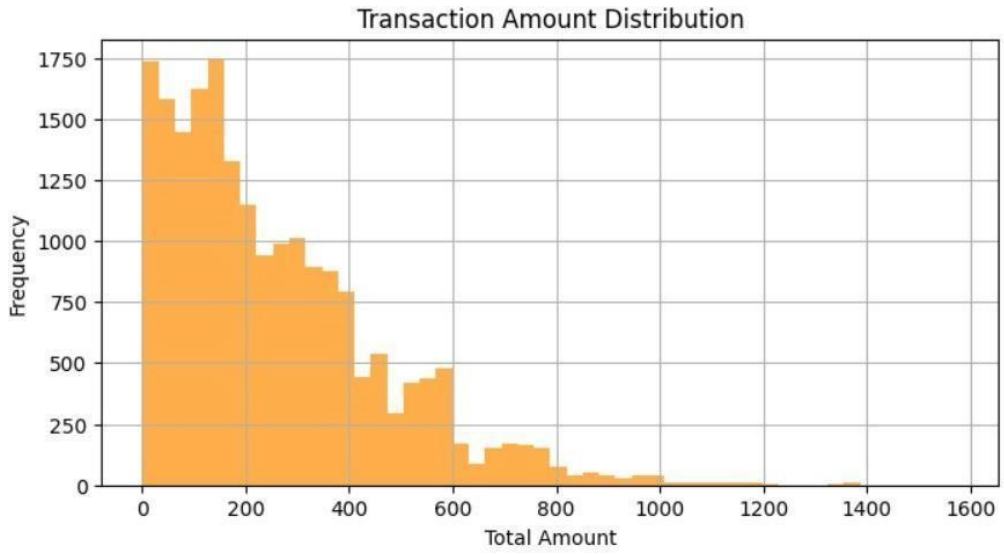

Figure 9: Total Amount Distribution

3.4.8 Outliers Box Plot

Flags extreme values in the key numeric variables that could distort analysis or modeling.

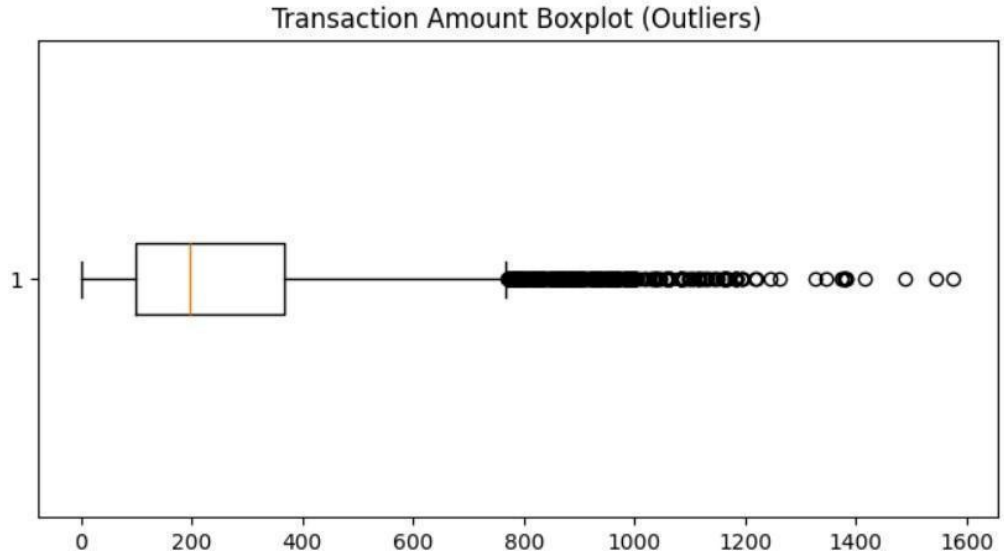

Figure 10: Outlier Detection

3.4.9 Daily Sales Trend

Follows sales day by day to expose short-term movements and fluctuations.

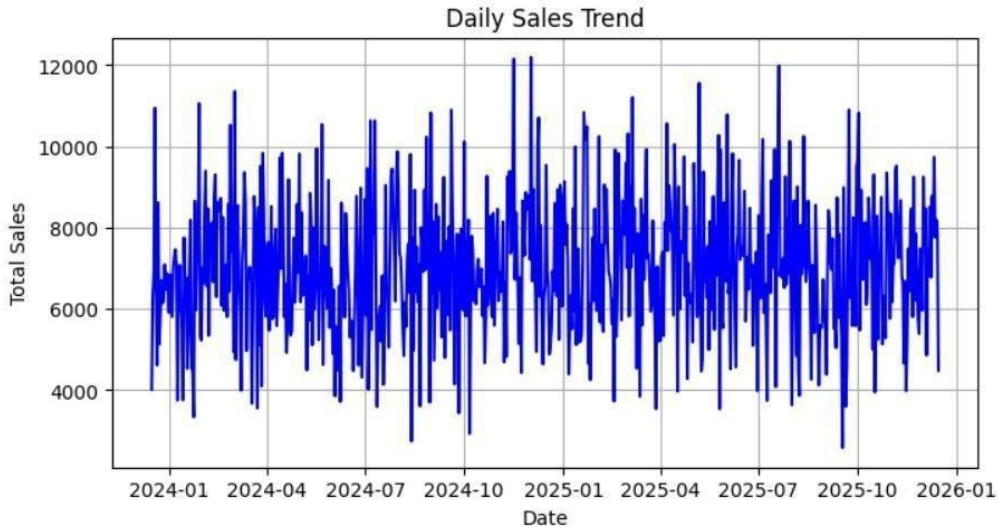

Figure 11: Daily Sales Trend

3.4.10 Monthly Sales Trend

Aggregates sales by month to reveal seasonality and longer-term growth.

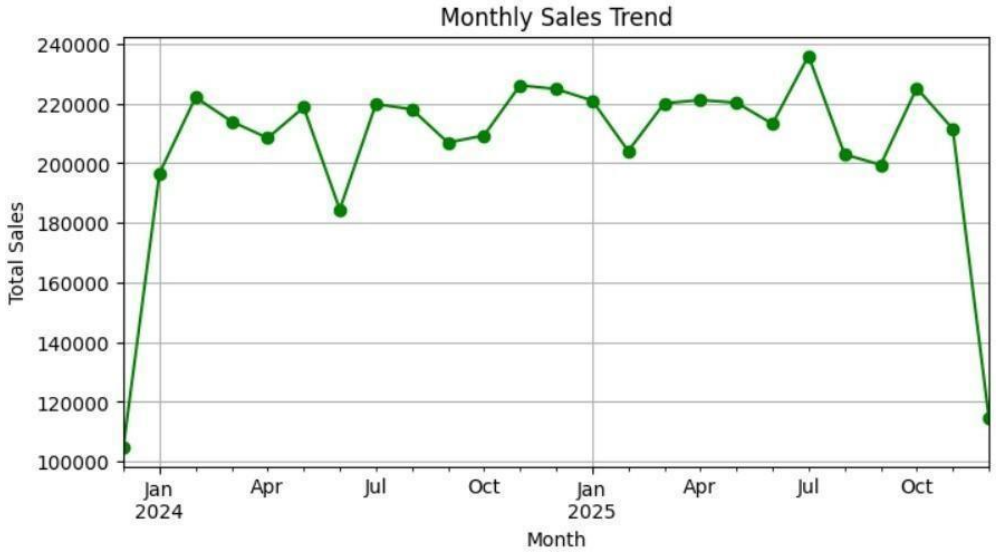

Figure 12: Monthly Sales Trend

3.4.11 Average Sales by Weekday

Contrasts average sales across weekdays to find the strong and weak days.

Figure 13: Average Sales by Weekday

3.4.12 Average Transaction Amount by Category

Measures average spend per transaction within each product category.

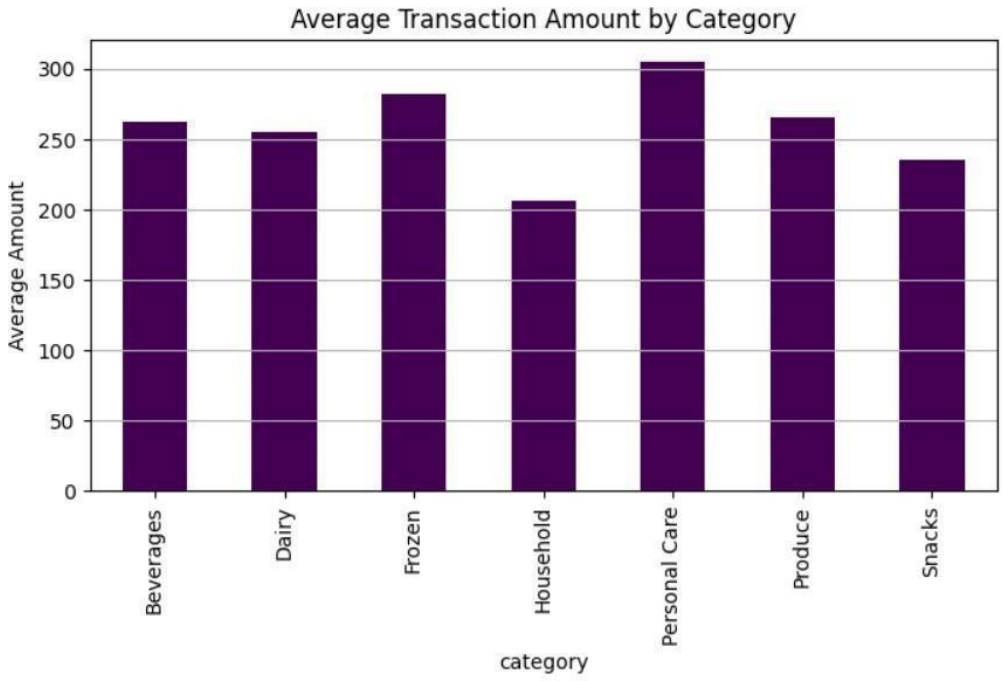

Figure 14: Average Transaction Amount by Category

3.4.13 Top 10 Stores by Sales

Ranks stores by total sales volume to spotlight the best performers.

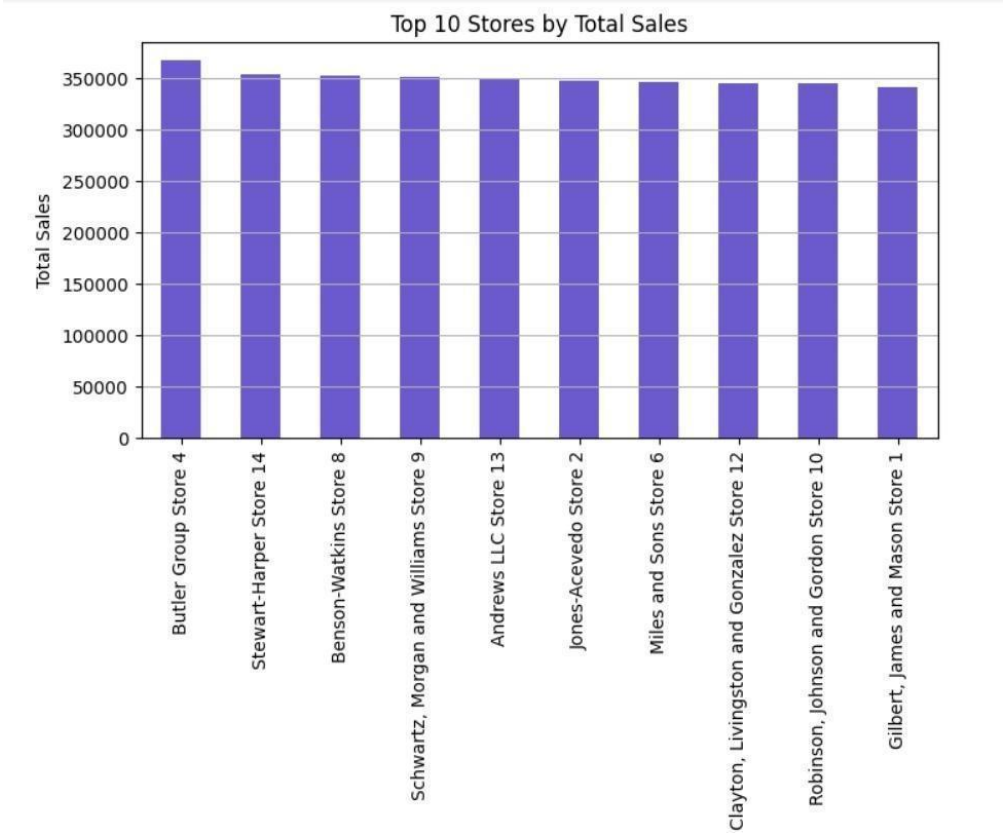

Figure 15: Top 10 Stores by Sales

3.4.14 Discount Vs Total Amount

Explores how offered discounts relate to the final transaction value.

Figure 16: Discount vs Total Amount

3.4.15 Quantity Vs Total Amount

Assesses how the quantity purchased drives the total transaction value.

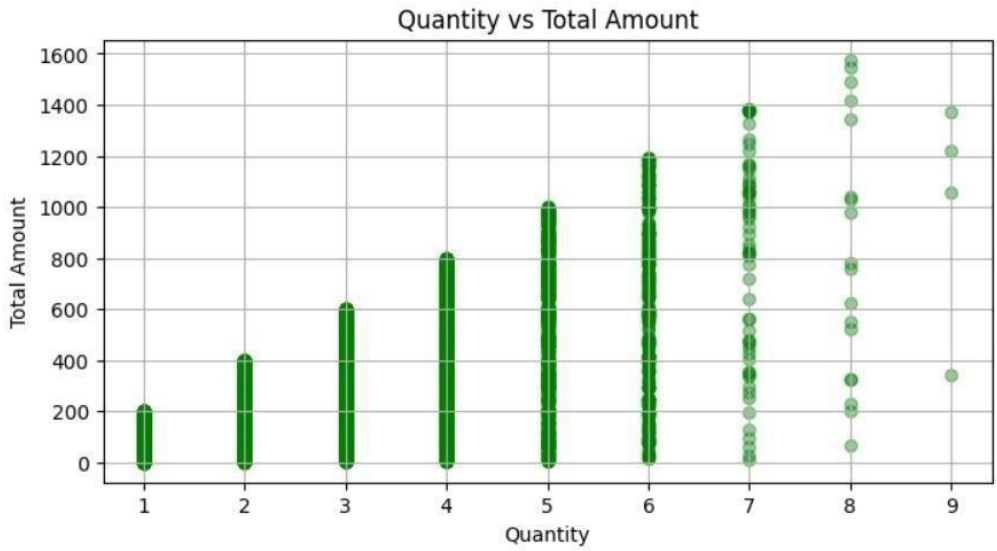

Figure 17: Quantity vs Total Amount

3.4.16 Correlation Matrix

Quantifies pairwise relationships among numeric variables to expose strong correlations and dependencies.

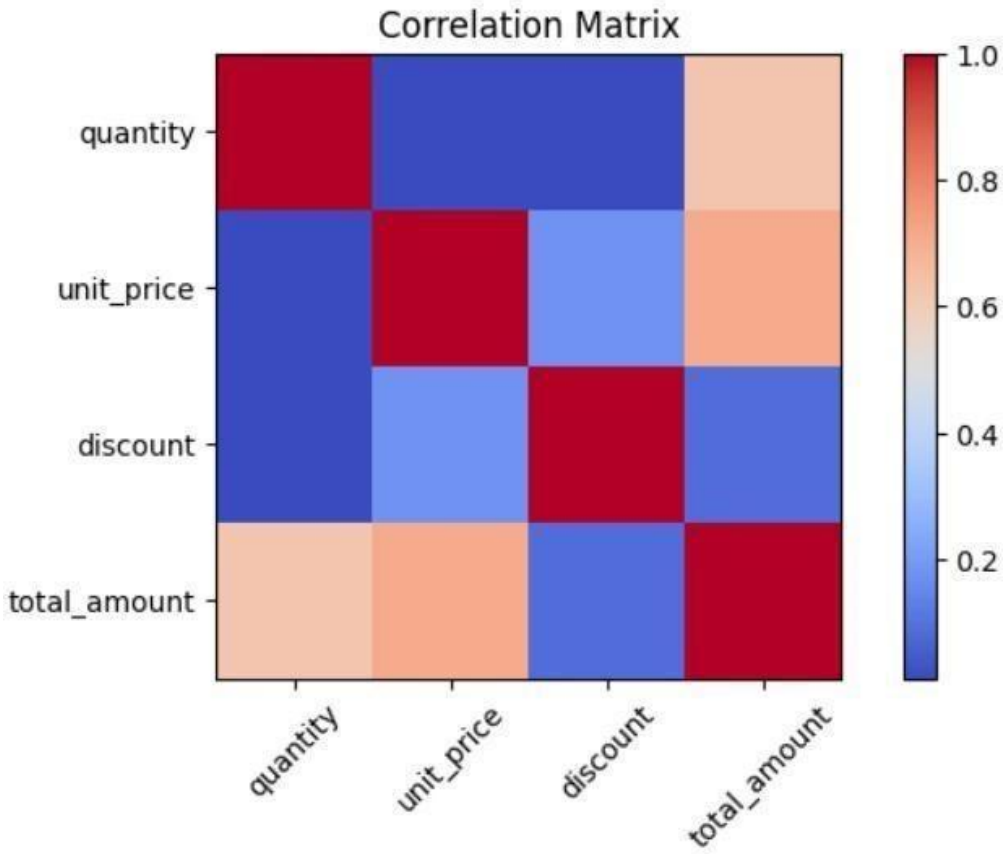

Figure 18: Correlation Heatmap

3.5 Functional Requirements

Table 3.1: Functional Requirements

Req ID	Functional Requirement	Description
FR1	Dataset Upload	Accept CSV and Excel file uploads
FR2	Dataset Preview	Show dataset structure and sample rows
FR3	Natural Language Query	Answer conversational questions about the data
FR4	Automated EDA	Produce statistics, correlations, and summaries
FR5	Visualization Generation	Build charts automatically from queries
FR6	Predictive Modeling	Train classification and regression models
FR7	Explainable AI	Explain predictions via SHAP with a deterministic fallback
FR8	Result Export	Offer HTML/PDF report downloads
FR9	Domain Validation	Detect and decline datasets outside the retail-mart domain
FR10	Verifiability	Attach computation receipts and a SHA-256 reproducibility certificate

3.6 Non-functional Requirements

Table 3.2: Non-functional Requirements

Req ID	Category	Description
NFR1	Usability	Simple UI for non-technical users
NFR2	Performance	Handle medium-sized datasets efficiently
NFR3	Reliability	Stable results across repeated queries
NFR4	Scalability	Modular design open to future extension
NFR5	Security	JWT authentication, per-user data isolation, rate limiting, safe file handling
NFR6	Maintainability	Clean modular backend covered by automated tests

3.7 Database Schema

Persistence runs on Supabase, a managed PostgreSQL service bundled with authentication and file storage. The tables below mirror what dataverse_backend/supabase_schema.sql actually creates. user accounts live in Supabase Auth (auth.users), and two storage buckets (dataverse-datasets, dataverse-reports) hold uploaded files and generated reports.

Table 3.3: Supabase Database Schema

Table Name	Key Attributes	Description
chat_sessions	id, user_id, title, status, active_dataset_id, created_at	One analysis workspace/session per user
chat_messages	id, session_id, role, content, message_type, payload	Conversation history with rich payloads
datasets	id, session_id, user_id, filename, storage_path, row_count, column_count, schema_profile, semantic_map	Dataset metadata, profile, and semantic mapping
agent_runs	id, session_id, dataset_id, agent_name, status, input, output	One record per agent execution, for traceability
reports	id, session_id, dataset_id, title, format, storage_path, metadata	Generated HTML/PDF report artifacts

3.8 Chapter Summary

This chapter set out the research problem, posed the research questions, fixed the objectives, bounded the scope, outlined the method, and listed the evaluation criteria. The gaps in the 2022 to 2025 literature argue for a unified conversational BI system joining EDA, visualization, prediction, and explainability inside one language-driven workflow. The chapters that follow describe how DataVerse AI was designed and implemented on that foundation.

Chapter 4:

Research Methodology

4.1 Introduction

4.1.1 Purpose

This Software Requirements Specification (SRS) collects every functional, non-functional, and system requirement for DataVerse AI, a conversational Business Intelligence platform that lets non-technical people run Exploratory Data Analysis (EDA), view findings, build predictive models, and read clear AI output through plain-language interaction. The document serves the development team as a technical contract and gives stakeholders, evaluators, and supervisors an authoritative reference.

4.1.2 Scope

DataVerse AI is a web application for analyzing data through conversation. A user types questions in ordinary language into an LLM-assisted interface, receives automated exploratory analysis, gets sensible chart suggestions, trains predictive models, and reads AI findings backed by verifiable receipts. Uploads and results flow through a chat-style interface. The frontend is Next.js 15 running React 19. the backend is FastAPI. The Python analytics engine draws on Pandas, Scikit-learn, SHAP, and ReportLab. Language capability comes from an OpenAI-compatible provider layer that can reach the OpenAI API, Google Gemini, Anthropic Claude, or the DeepAnalyze-8B agentic data-science model, and the product drops into a fully deterministic Mock mode whenever no provider is configured. An authenticated user can upload a dataset, ask analytical questions, and collect charts, findings, and predictive explanations without writing a line of code.

4.1.3 Intended Audience

Several groups benefit from the application. Students and researchers can question academic datasets conversationally to support their work. Analysts and managers can study data without SQL, Python, or conventional BI tooling. Supervisors and evaluators can inspect accuracy, documentation quality, and compliance. Developers can treat this SRS as the blueprint for the structure, APIs, interface, and analytics engine.

4.1.4 Definitions, Acronyms, and Abbreviations

BI, Business Intelligence, is the practice of examining data to support decisions. EDA, Exploratory Data Analysis, uncovers patterns, trends, and anomalies. LLM, Large Language Model, denotes an AI model that reads and writes human-like text. NL, Natural Language, is everyday human phrasing used to operate a system. NL2SQL is the conversion of natural language questions into SQL statements. XAI, Explainable Artificial Intelligence, keeps AI decisions transparent and understandable. API, Application Programming Interface, is the contract through which software components communicate. UI/UX covers interface design and the experience of using it. CSV, Comma-Separated Values, is a common tabular file format. JWT, JSON Web Token, is the credential format used for authenticated API access.

4.2 Overall Description

4.2.1 Product Perspective

DataVerse AI stands alone as a web-based conversational analytics product. The frontend combines a chat UI, an upload module, and visualization rendering. The backend houses a language interpretation service backed by the optional LLM provider layer, the Python analytics engine, a chart generator, a model builder, and explanation modules based on SHAP with a deterministic feature-importance fallback. Supabase supplies authentication, PostgreSQL persistence, and file storage, and a local session cache accelerates repeated analysis. Nothing external is strictly required: with no LLM configured, the product still runs end to end in deterministic Mock mode. In outline, the user talks to the frontend. the frontend calls the FastAPI backend. the backend runs the analytics engine (and the LLM when available), assembles results, and returns them.

4.2.2 Product Functions

Functionality is organized in modules. Language interpretation turns a request such as showing missing values into the corresponding EDA, visualization, or ML task. Users upload CSV or Excel files and preview structure: sample rows, column names, and types. A semantic mapper labels each upload (for instance mart_sales or retail_sales), and a domain guard turns away datasets that fall outside the retail-mart scope, explaining the refusal. Automated EDA produces summary statistics, missing-value analysis, outlier detection, correlation matrices, and

descriptive findings. The visualization module proposes fitting chart types, bar charts, trend lines, donut charts, histograms, heatmaps, and writes captions and takeaways automatically. Predictive modeling trains regression or classification models (Ridge and Random Forest) with automatic pipeline selection and reports metrics such as Accuracy, F1-score, and RMSE. The explainability module, SHAP plus its deterministic fallback, renders feature importance, runs counterfactual what-if analysis, and writes plain-language summaries. Everything is reachable from the chat interface across multiple turns, with follow-up questions and embedded visuals. Every KPI exposes a receipt showing its calculation, and each analysis is fingerprinted with SHA-256 so it can be re-verified against the raw data.

4.3 User Characteristics

Three user groups are anticipated. Non-technical users, business managers and students, need only basic computer literacy. Semi-technical users are analysts acquainted with BI tools and elementary statistics. The technical group, developers, should know Python, FastAPI, JavaScript frameworks, and the general shape of ML workflows.

4.4 Constraints

Certain limits apply. Reaching the hosted frontend and the Supabase services requires an internet connection, and the optional narration layer additionally needs a configured LLM provider. The product is browser-only. no desktop or mobile client exists. Throughput and scale are bounded by the host server. Uploads are limited to 50 MB per dataset, and all file handling must be secure to protect user data.

4.5 Assumptions and Dependencies

A few assumptions and dependencies underpin the application. Uploads are expected to be valid, well-formed CSV or Excel files from the retail-mart domain. The LLM services (OpenAI, Gemini, Claude, or DeepAnalyze) are strictly optional: whenever they are absent the application produces deterministic narration, so core analytics never depend on them. Correct installation of the Python stack, Pandas, Scikit-learn, and SHAP, is required, along with a configured Supabase project covering authentication, PostgreSQL, and storage. Stable backend hosting completes the list, ensuring consistent performance and availability.

4.6 Specific Requirements

4.6.1 Functional Requirements

The product must satisfy a set of core functional requirements. Users must be able to register, confirm an email address, and sign in, upload datasets, and have file formats such as CSV and XLSX validated. Analytical questions must be classified into EDA, visualization, or machine learning tasks. Exploration must run automatically: summary statistics, missing values, outliers, and correlation matrices, delivered in both visual and textual form. Each upload must be checked against the retail-mart domain and refused with an explanation when it does not belong. Chart suggestions must follow the data types and render inside the chat. Model training for classification or regression must respond to user prompts and report performance metrics. Explainability must compute SHAP or feature-importance attributions and present them visually. Finally, the conversation must span multiple turns so users can ask follow-up questions naturally.

4.6.2 Non-functional Requirements

Quality attributes matter as much as features. Performance: EDA on a medium dataset should finish within 5 to 10 seconds, and model training within a reasonable window. Usability: the interface stays minimal and intuitive, with clearly labeled, readable charts. Reliability: deterministic execution suppresses LLM hallucination and keeps results consistent across sessions. Security: datasets stay private per user, enforced by JWT authentication, per-user isolation, and API rate limiting, with all input sanitized against injection. Scalability: several analysis sessions must run concurrently, and the backend must be able to scale horizontally under cloud deployment.

4.6.3 External Interface Requirements

Interfaces are defined for users and for software. The UI offers a chat window, an upload panel, chart display components, and EDA summary tables. No special hardware is needed. ordinary laptops and phones suffice. The software boundary comprises the FastAPI backend, the Next.js frontend, Supabase services, and an optional LLM API, all speaking JSON over HTTPS for structured, secure exchange.

4.7 Validation and Acceptance Criteria

Acceptance rests on several checks. Every functional requirement must clear unit and integration testing. the automated backend suite holds 214 tests. Query interpretation accuracy must meet the defined bar. EDA results and charts must agree with independent Python analysis. SHAP and feature-importance explanations must be reviewed for correctness. User Acceptance Testing should return a System Usability Scale (SUS) score of 70 or above, and the Requirements Traceability Matrix must tie each requirement to its test cases for full coverage.

4.8 Appendices

Development and documentation rely on a defined toolset. Next.js with Tailwind CSS v4 builds the frontend, deployed on Vercel. FastAPI powers the backend services, exposed publicly through an ngrok secure tunnel in the demo deployment. Python with Pandas, Scikit-learn, SHAP, and ReportLab covers analysis, machine learning, explainability, and PDF rendering. Supabase supplies authentication, PostgreSQL persistence, and file storage. Docker images are provided for containerized deployment of both tiers. GitHub Actions runs continuous integration and pytest drives the automated tests. Figma hosts the UI/UX design work, and MS Word carries the documentation.

CHAPTER 5

Proposed theoretical model and Software Design Specification

5.1 Design Methodology and Software Process Model

DataVerse AI was designed with a modular, iterative engineering approach suited to AI systems that demand constant refinement and testing. The process model is a hybrid of the Incremental Model and Agile practice: conversational processing, exploratory analysis, visualization, predictive modeling, and explainable AI were each designed, built, and evaluated as separate increments while staying integrated in one system.

Working incrementally means each module is validated before the next begins, containing complexity and risk. Agile habits allow conversational workflows and UI/UX decisions to be reshaped by testing and feedback. The combination fits this project because requirements evolve, AI components need experimentation, and the frontend, backend, and analytics layers interact closely.

5.2 System Overview

DataVerse AI is a web-based conversational Business Intelligence system that lets non-technical users complete entire analytical workflows in natural language: upload a dataset, ask questions, and receive automated exploration, charts, predictions, and explainable output.

Three layers compose the application: presentation for interaction, application for logic and orchestration, and analytics for computation and modeling. Clean APIs connect the layers, preserving scalability, maintainability, and clarity.

5.2.1 Architectural Design

The implementation uses a small two-agent path. After the signed-in user submits a file or question, Next.js forwards the request to FastAPI. FastAPI checks the request and coordinates DatasetAgent and AnalystAgent. The agents call Pandas, scikit-learn, SHAP, and the report builder for deterministic computation. FastAPI uses the separate Supabase connection for authentication, metadata, datasets, and saved reports. The verified charts and reports are then shown in the user output stage.

Architecture Diagram

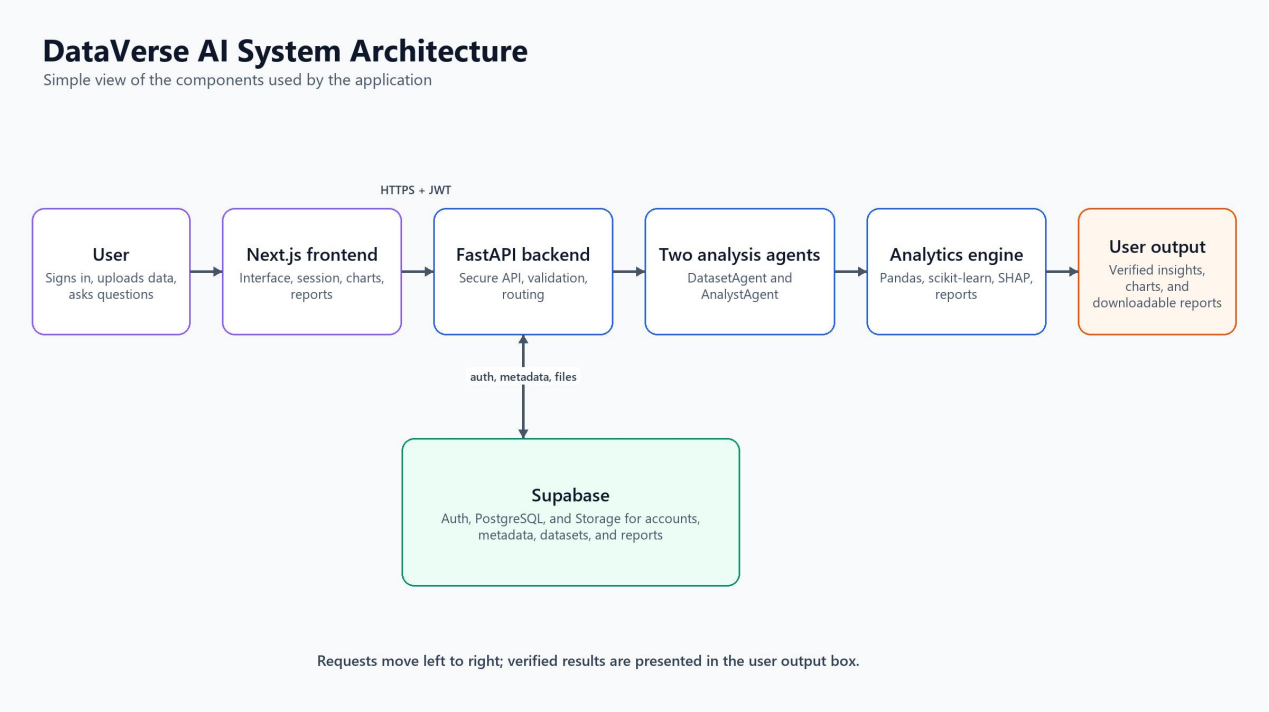

Figure 19: Updated DataVerse AI Architecture

5.2.2 Flowchart Description and Component Explanation

Figure 20 separates the account flow from the analysis flow. Sign-up, login, refresh, and email confirmation go to Supabase Auth through FastAPI. Dataset uploads and questions go to SessionService, then to the two agents and the deterministic analytics modules. Metadata is written to Supabase PostgreSQL, while dataset and report files are written to private Storage buckets. The response travels back through FastAPI to the Next.js interface.

Flowchart:

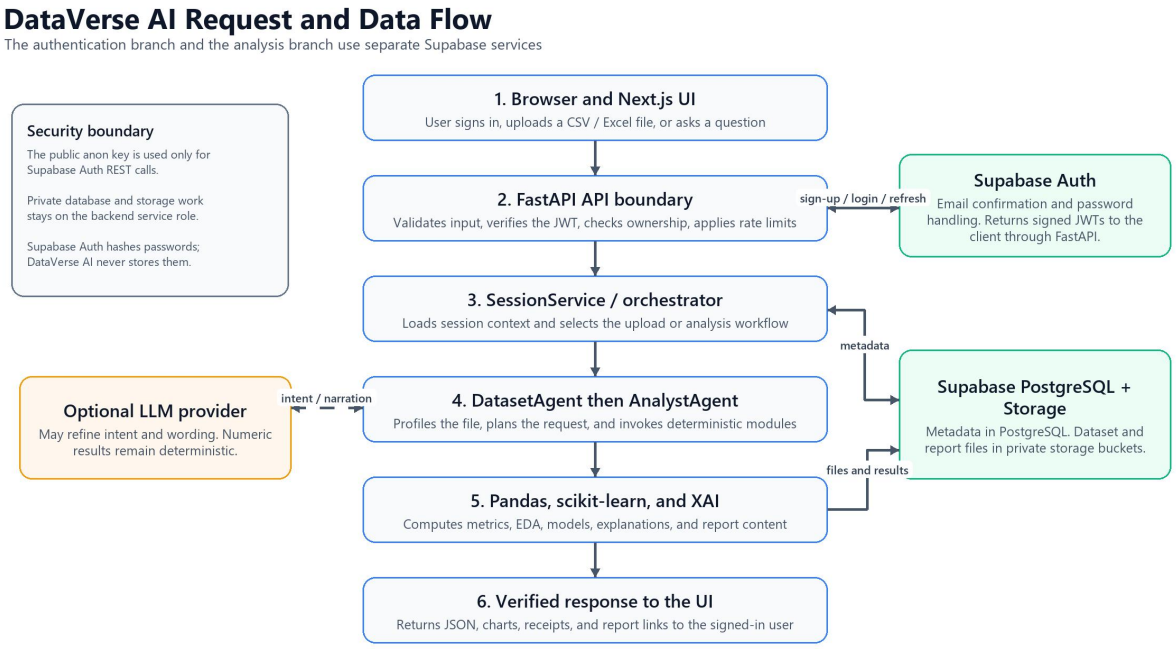

Figure 20: System Flowchart

1. Browser and Next.js UI

The signed-in user uploads a CSV or Excel file and asks questions in the Next.js interface. The browser keeps only the session state and sends HTTPS requests through the same-origin backend proxy.

2. FastAPI API Boundary

FastAPI validates request data, verifies the Supabase JWT, checks resource ownership, applies rate limits, and returns security headers before any analysis or storage operation runs.

3. SessionService and Orchestrator

SessionService loads the active session, messages, and dataset metadata. It then selects the upload, question, analysis, or report workflow and records the resulting agent run.

4. DatasetAgent

DatasetAgent checks the file extension and size, parses the upload, normalizes column names, profiles the DataFrame, and records data-quality results before analysis begins.

5. AnalystAgent

AnalystAgent turns the request into an analysis plan and calls the deterministic modules for KPIs, EDA, prediction, explanation, and report composition. It does not invent numeric results.

6. Deterministic Analytics Engine

Pandas computes the statistics, business metrics, trends, correlations, and outliers. Scikit-learn trains supported models, while SHAP or feature importance explains model behavior.

7. Supabase Auth

Supabase Auth owns password hashing, email confirmation, login, token refresh, and JWT issuance. DataVerse AI never stores raw passwords in its database or application code.

8. Supabase PostgreSQL

The backend service role writes session, message, dataset, agent-run, and report metadata to PostgreSQL. Row ownership is always checked against the authenticated user.

9. Supabase Storage

Private Storage buckets hold the uploaded dataset files and generated reports. Only the backend uses the service-role credential for these operations.

10. Optional LLM Provider

An optional language model may refine intent or improve wording. It is outside the deterministic calculation path and therefore cannot supply a KPI, model metric, or chart value.

11. Verified Response and Reports

FastAPI returns JSON results, charts, receipts, and report links to the Next.js interface. The response is shown only in the authenticated user's workspace.

5.3 Design Models

The design models show how components talk and how data moves. Box-and-line diagrams map the user, the frontend, the backend services, the agents, the analytics engine, and the database, underlining the split between conversational reasoning and analytical execution.

The chosen style is a service-oriented, layered architecture with each major capability wrapped in its own module, so conversation, analytics, visualization, and explainability can evolve independently yet operate as one workflow.

5.4 Data Design

Data design targets efficient storage, processing, and retrieval of structured analytical data. The product works with tabular CSV and Excel uploads. During processing, datasets are cached locally for the session while metadata and results persist in Supabase for traceability.

Entities exist to serve analytical workflows rather than long-term transactions. The relationships among sessions, messages, datasets, agent runs, and reports are explicit, which keeps results consistent and reproducible.

5.4.1 Data Dictionary

The data dictionary describes the principal entities. Supabase Auth (auth.users) owns user accounts. chat_sessions holds each analysis workspace with its owner and active dataset. chat_messages preserves the conversation, including rich payloads such as charts and KPIs. datasets captures file name, storage path, row and column counts, the schema profile, and the semantic map. agent_runs logs every agent execution with inputs and outputs for traceability, and reports stores generated HTML/PDF artifacts. The structure keeps the design clear, traceable, and maintainable.

5.5 User Interface Design

The current interface uses four public screens: the landing page, the feature overview, secure login, and secure sign-up. Guest access has been removed. New accounts require a 12-character password with mixed character classes and must be confirmed through the email link sent by Supabase Auth.

Upload panels, conversational replies, embedded charts, and explainability visuals share one unified view. The design observes standard UI/UX principles: low cognitive load, readable charts, and clear system feedback.

5.6 Screens

Current public pages

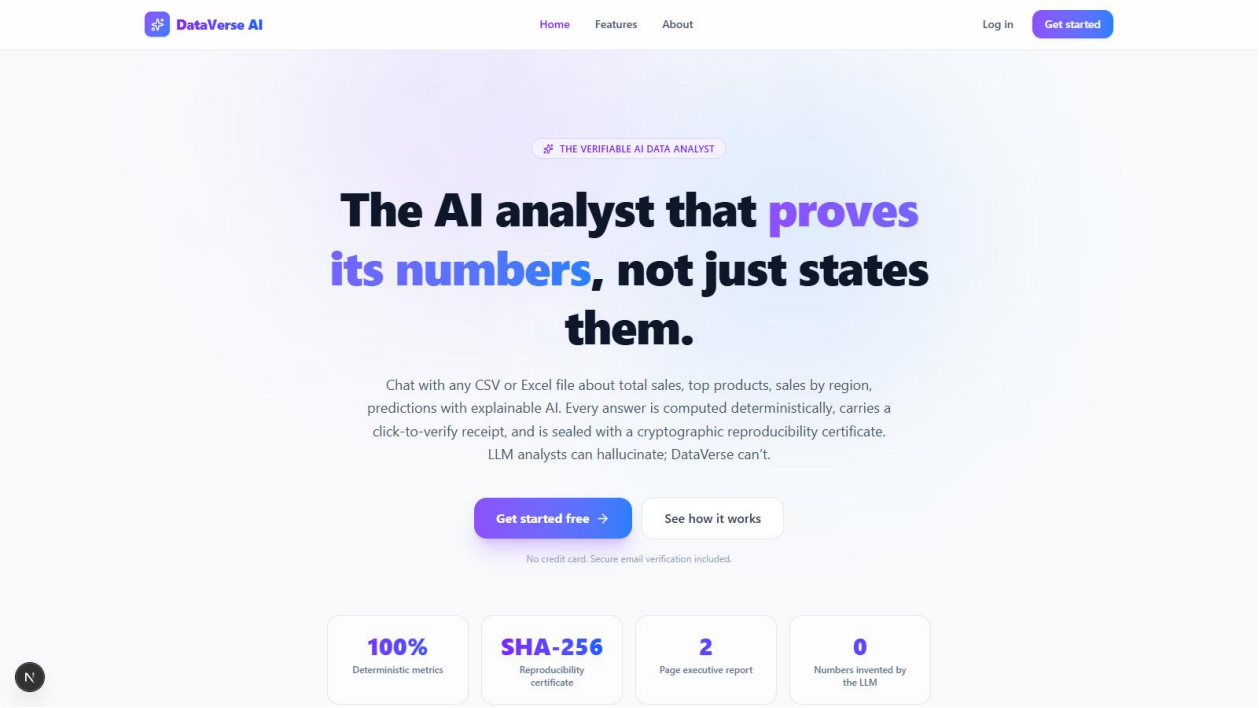

Figure 21: Current Landing Page

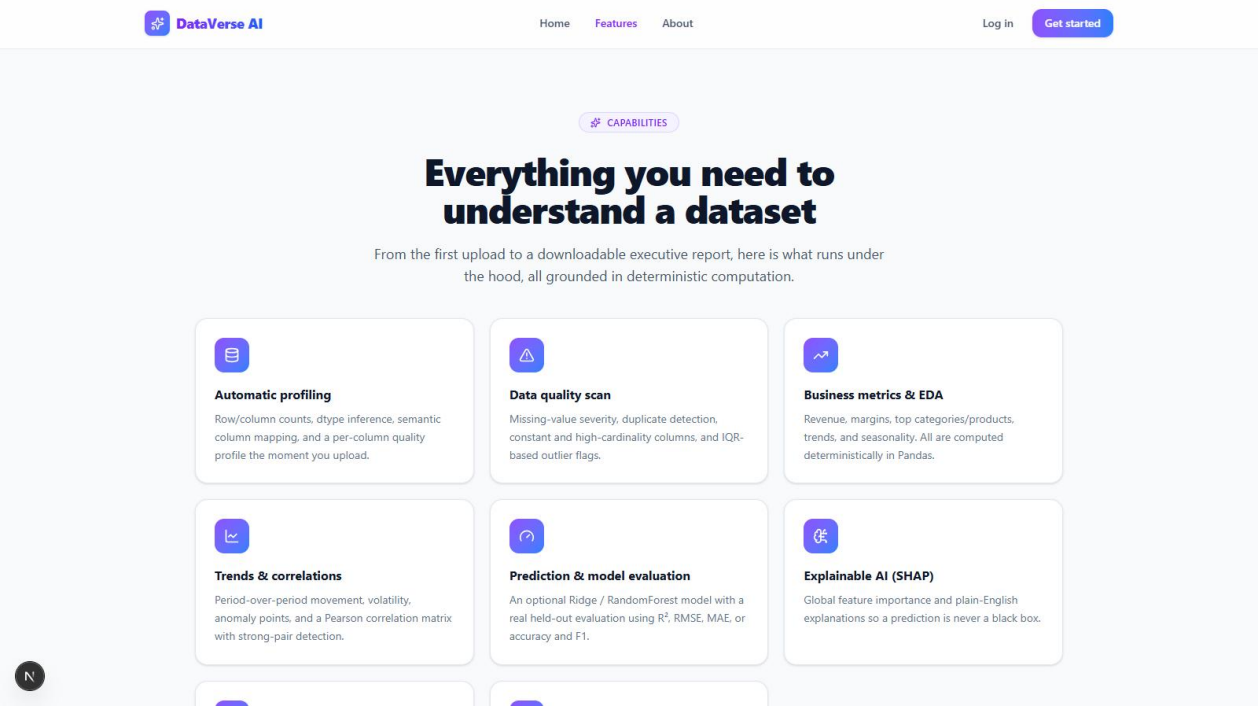

Figure 22: Current Features Page

Secure account access

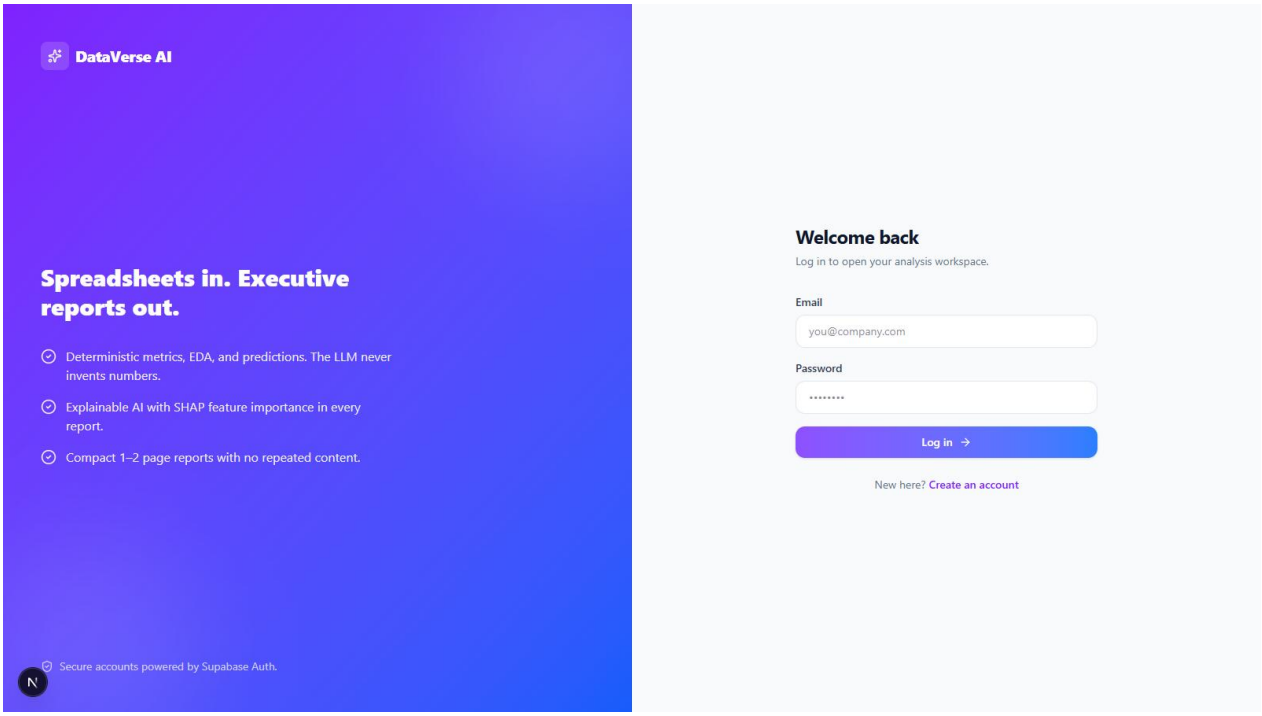

Figure 23: Secure Login Page

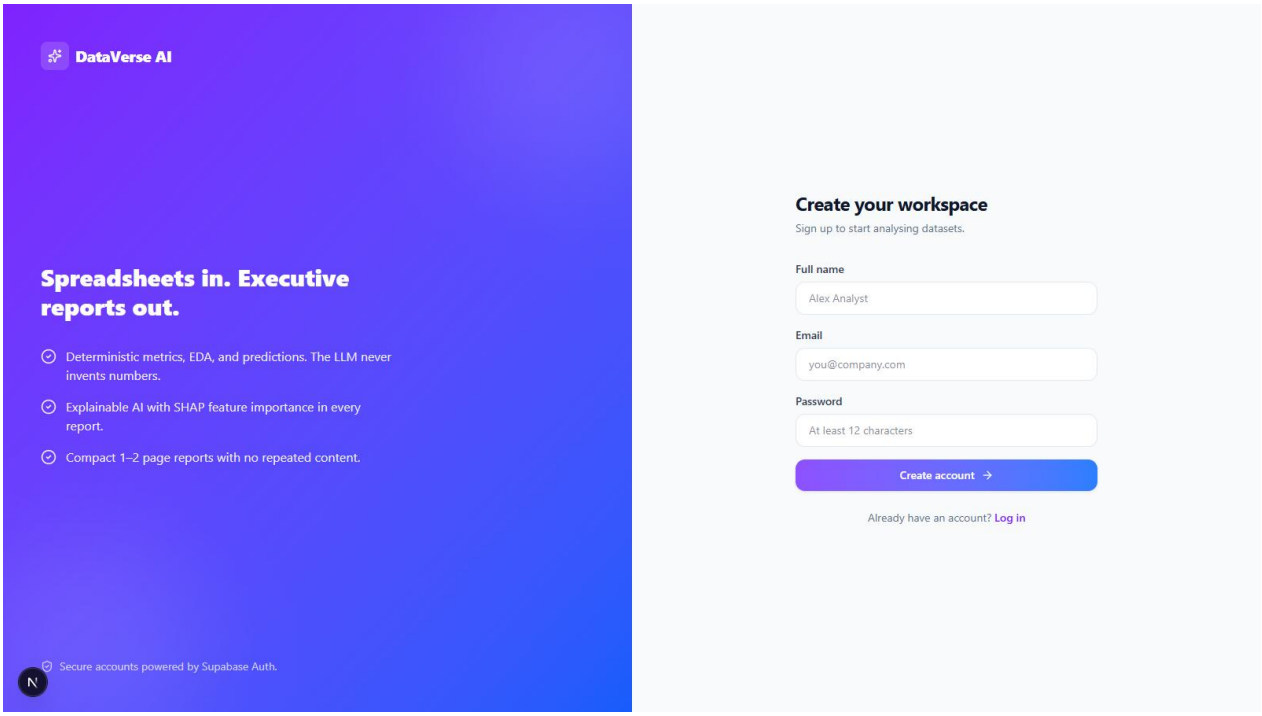

Figure 24: Secure Sign-up Page with Email Confirmation

Chapter 6

DATA COLLECTION & ANALYSIS

6.1 Introduction

Here we document the data collection and analysis phase of DataVerse AI. The product accepts CSV or Excel uploads and returns automated profiling, findings, charts, recommendations, predictions, and downloadable reports. The pages below describe the evaluation dataset, how it is loaded, the preprocessing applied, the quality checks performed, and the exploratory analysis the application carried out.

The dataset behind Chapter 6 and Chapter 7 is a processed retail mart file, retail_mart_processed_v1.csv. It records retail transactions and covers sales, profit, discount, quantity, customer type, payment method, online/offline channel, store, region, city, category, and several time fields.

6.2 Dataset Source and Collection

The file lives inside the project repository at data/retail_mart_processed_v1.csv and holds 35,000 rows across 21 columns. A row is one retail sale. the columns describe the store, its location, the product category, pricing, quantity, discount, sales, profit, customer type, payment method, channel, and time attributes.

This dataset was chosen because it can answer a range of realistic business questions: the top-selling category, the best-performing region, total profit, which customer segment brings more revenue, how online compares with offline, what discounts do to profit, which stores lead, and how sales move over time.

Table 6.1: Dataset Profile

Attribute	Description
Dataset Name	retail_mart_processed_v1.csv
Dataset Path	data/retail_mart_processed_v1.csv
Number of Rows	35,000
Number of Columns	21
Dataset Type	Retail transaction dataset
Main Business Focus	Sales, profit, discount, store, customer, category, and channel analysis
Important Target Columns	total_sales, profit, quantity, profit_margin, discount

Time Columns	weekday, month, year, hour

6.3 Dataset Loading Process

Uploads arrive through the frontend: the user selects a CSV or Excel file and the browser sends it to the FastAPI backend for validation, parsing, profiling, and analysis. The upload interaction lives in frontend/app/page.tsx, and the endpoint in dataverse_backend/app/api/session_routes.py.

On receipt, the backend confirms the file type, checks the size limit, and forwards the bytes to the parser. The logic in dataverse_backend/app/api/upload_parsing.py reads CSV and Excel through Pandas. The loaded frame is then semantically mapped, screened by the retail-domain guard, and handed to the analysis pipeline for profiling and analysis.

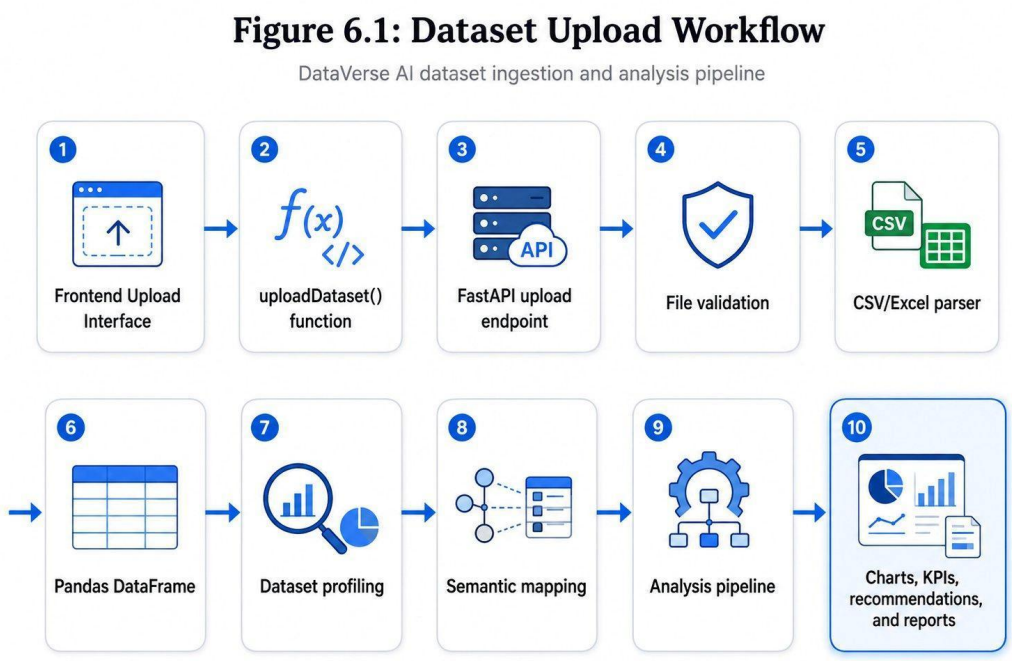

Figure 6.1: Dataset Upload Workflow

6.4 Dataset Description

The retail file carries 21 fields. Many are numeric on the surface, yet several encode categories: region, city, category, customer type, payment method, and the online order flag.

Table 6.2: Dataset Columns and Analytical Role

Column Name	Data Type	Analytical Role
store_id	Integer	Store identifier
region	Integer	Encoded region category
city	Integer	Encoded city category
category	Integer	Encoded product category
subcategory	Integer	Encoded product subcategory
unit_price	Float	Product unit price
quantity	Integer	Quantity sold
discount	Float	Discount percentage
total_sales	Float	Total sales value
profit	Float	Profit value
customer_type	Integer	Encoded customer segment
payment_method	Integer	Encoded payment method
online_order	Integer	Online/offline order flag
stockout_flag	Integer	Stock availability flag
weekday	Integer	Day of week
month	Integer	Month
year	Integer	Year
hour	Integer	Hour of transaction
price_qty	Float	Price multiplied by quantity
discount_value	Float	Discount amount
profit_margin	Float	Profit margin

6.5 Data Preprocessing

Although the file arrived already structured, DataVerse AI still runs a series of preprocessing and validation steps so the data is readable, complete, and fit for business intelligence. The sequence is: (1) file type validation. (2) file size validation. (3) CSV/Excel parsing. (4) empty dataset validation. (5) semantic mapping of columns to business meanings. (6) retail-domain validation. (7) data type inference. (8) missing value detection. (9) duplicate row detection. (10) outlier detection with the Interquartile Range method. and (11) execution of the analysis pipeline.

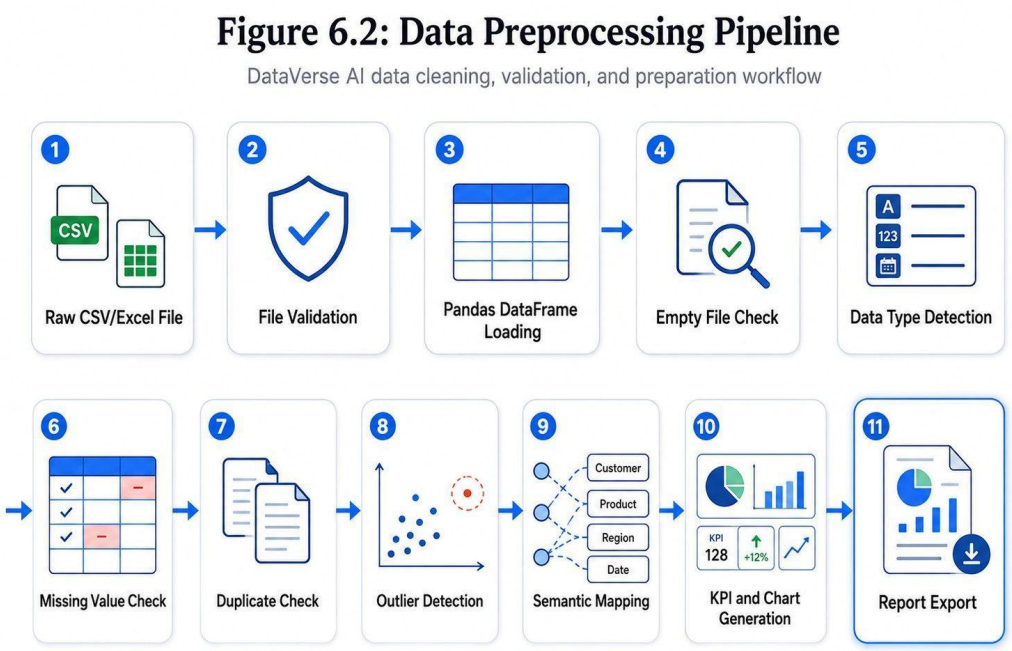

Figure 6.2: Data Preprocessing Pipeline

6.6 Data Quality Analysis

Quality analysis verified the dataset's completeness and reliability. No missing values and no duplicate rows were found, so neither imputation nor deduplication was necessary.

Table 6.3: Data Quality Summary

Quality Metric	Result
Total Rows	35,000
Total Columns	21
Missing Values	0
Missing Percentage	0.00%
Duplicate Rows	0
Constant Columns	None detected
Data Quality Score	1.0
Dataset Status	Suitable for analysis

With nothing missing and nothing duplicated, the file was ready for exploration as-is. One caveat: several categorical fields are stored as numeric codes, so the application can point to category 1 or region 2 but cannot print human-readable names unless mapping tables are supplied.

6.7 Exploratory Data Analysis

Exploration examined sales, profit, quantity, customers, payments, channels, discounts, and store-level behavior. Table 6.4 summarizes the headline business results.

Table 6.4: Main Business KPIs

KPI	Value
Total Sales	$2,248,662.62
Total Profit	$336,938.93
Total Quantity Sold	69,546
Average Sales per Row	$64.25
Overall Profit Margin	14.98%
Best Region by Sales	Region 2
Best Category by Sales	Category 1
Best Category by Profit	Category 1
Best Store by Sales	Store 27

6.7.1 Sales by Region

At region level, region 2 dominates with $1,279,763.60 in sales and $191,713.72 in profit over 19,905 orders, marking it the strongest market segment in the file.

Table 6.5: Sales by Region

Region	Total Sales	Total Profit	Orders	Profit Margin
2	$1,279,763.60	$191,713.72	19,905	14.98%
0	$328,285.13	$49,646.34	5,051	15.12%
1	$320,928.87	$47,777.62	5,059	14.89%
3	$319,685.02	$47,801.25	4,985	14.95%

6.7.2 Sales and Profit by Category

By category, category 1 leads on both measures, contributing $964,022.78 in sales and $144,405.95 in profit across 8,794 orders.

Table 6.6: Category Performance

Category	Total Sales	Total Profit	Orders	Profit Margin
1	$964,022.78	$144,405.95	8,794	14.98%
2	$433,278.77	$65,192.01	8,826	15.05%
3	$426,792.73	$63,601.16	8,685	14.90%
0	$424,568.34	$63,739.81	8,695	15.01%

6.7.3 Customer Type Analysis

Customer type 0 edges ahead with $1,136,762.84 in sales and $170,677.48 in profit versus customer type 1. The margin between the groups is slim, meaning both segments matter nearly equally to the business.

Table 6.7: Customer Type Analysis

Customer Type	Total Sales	Total Profit	Orders	Profit Margin
0	$1,136,762.84	$170,677.48	17,522	15.01%
1	$1,111,899.78	$166,261.45	17,478	14.95%

6.7.4 Payment Method Analysis

Among payment methods, method 2 tops sales at $569,968.83 while method 3 tops profit at $85,219.69. Figures sit close together across all four methods, indicating balanced channel usage.

Table 6.8: Payment Method Analysis

Payment Method	Total Sales	Total Profit	Orders	Profit Margin
2	$569,968.83	$85,175.19	8,766	14.94%
3	$563,971.83	$85,219.69	8,845	15.11%
1	$560,863.06	$83,464.06	8,742	14.88%
0	$553,858.90	$83,079.99	8,647	15.00%

6.7.5 Online and Offline Order Analysis

The online_order flag permits a channel comparison. Online orders came in slightly ahead, $1,129,750.25 in sales and $169,789.49 in profit, against $1,118,912.37 and $167,149.44 for offline.

Table 6.9: Online vs Offline Sales

Online Order	Total Sales	Total Profit	Orders	Profit Margin
1 (Online)	$1,129,750.25	$169,789.49	17,570	15.03%
0 (Offline)	$1,118,912.37	$167,149.44	17,430	14.94%

6.7.6 Discount Impact Analysis

Discounting was analyzed against profit. The Pearson correlation between the two is -0.0739: a weak negative link, so deeper discounts coincide mildly with lower profit, but far too weakly to treat discounting as the deciding factor for profitability.

Table 6.10: Discount Impact on Profit

Discount	Total Sales	Total Profit	Avg. Profit	Orders
0.00	$803,438.05	$120,942.04	$10.45	11,568
0.05	$383,225.49	$57,605.96	$9.98	5,773
0.10	$377,970.56	$56,302.83	$9.50	5,929
0.15	$351,711.37	$52,771.19	$9.01	5,854
0.20	$332,317.15	$49,316.91	$8.39	5,876

6.8 Tools and Technologies Used

Data collection and analysis relied on the following tools.

Table 6.11: Tools and Technologies

Tool / Technology	Purpose
Python	Backend processing and data analysis
Pandas	CSV/Excel loading and DataFrame analysis
Scikit-learn	Predictive modeling (Ridge, Random Forest) and evaluation
SHAP	Model explainability, with a deterministic feature-importance fallback
FastAPI	Backend API development
Next.js 15 / React 19 + Tailwind CSS v4	Frontend interface and styling, deployed on Vercel
Hand-built SVG / Chart.js / ReportLab	Charts for the app, HTML reports, and PDF reports
Supabase	Authentication, PostgreSQL persistence, and file storage (required in production)
LLM provider layer	Optional narration and agent reasoning (OpenAI / Gemini / Claude / DeepAnalyze-8B); deterministic Mock mode without keys
ngrok	Secure tunnel exposing the demo backend to the deployed frontend
Docker	Containerization of the backend and frontend for deployment
pytest + GitHub Actions	Automated test suite (214 tests) and continuous integration
HTML/PDF Export	Downloadable report generation

6.9 Ethical Considerations

The dataset carries no direct personal identifiers: no names, emails, phone numbers, or addresses. Customer type and payment method appear only as numeric codes, which limits privacy exposure. The application keeps sensitive environment variables such as API keys out of reports and frontend code, guards uploads behind authenticated per-user access, and filters internal pipeline warnings out of user-facing reports. Should real business data be uploaded in future deployments, secure file handling, access control, retention policies, and confidentiality protections must all be maintained.

6.10 Summary

This chapter recorded the dataset's origin, its loading path, preprocessing, quality checks, and the exploratory analysis. The file spans 35,000 rows and 21 columns and suits retail business analysis well. Quality checks found no missing values and no duplicates. Exploration crowned region 2 the strongest sales region, category 1 the leader in both sales and profit, and store 27 the top store. Online sales run slightly ahead of offline, and discounting shows only a weak negative association with profit.

Chapter 7

RESULTS & DISCUSSION

7.1 Introduction

Here we report what happened when DataVerse AI was applied to the retail mart dataset. The question under evaluation is whether the application performs dataset upload, profiling, quality analysis, KPI generation, visualization, report generation, prediction, and integration correctly. The chapter also interprets the findings, sets the application against manual analysis, and records the limitations.

7.2 Experimental Setup

The experiment ran on the DataVerse AI repository. The frontend, Next.js 15 with React 19, is deployed on Vercel. the backend is FastAPI with Supabase supplying authentication, persistence, and storage. Evaluation used retail_mart_processed_v1.csv from the project's data directory. Checks covered dataset reading, pipeline execution, project file inspection, report generation, frontend build verification, linting, the automated test suite, and a live end-to-end exercise on the deployed site in which a 10,000-row retail dataset was uploaded through the browser and analyzed by the hosted backend.

7.3 Dataset Upload and Profiling Results

In the implemented version, the application read and profiled the dataset without error: the parser confirmed 35,000 rows and 21 columns. Upload accepts CSV and Excel, and the pipeline yields the dataset profile, semantic mapping, business metrics, charts, and report outputs. In the live deployment test, a 10,000-row, 7-column retail file parsed in 94 ms, profiled in 281 ms, and was classified as mart_sales in 15 ms.

Table 7.1: Dataset Upload and Profiling Results

Test Case	Expected Result	Actual Result	Status
CSV file loading	Dataset loads into a DataFrame	35,000 rows and 21 columns loaded	Passed
Dataset profiling	Columns and types identified	Column profile generated	Passed
Semantic mapping	Dataset type detected	Classified as retail mart sales	Passed
Domain validation	Non-retail data refused	Medical test dataset refused	Passed

		with a clear message	
Missing value check	Missing cells counted	0 missing values found	Passed
Duplicate check	Duplicate rows counted	0 duplicate rows found	Passed
HTML report generation	HTML report created	HTML generation successful	Passed
PDF report generation	PDF report created	PDF generated in the 2-page executive format	Passed

7.4 Data Quality Results

Quality results were clean: every column free of missing values, zero duplicate rows. The dataset needed no imputation and no deduplication before analysis.

Table 7.2: Data Quality Results

Metric	Result
Rows	35,000
Columns	21
Missing Values	0
Missing Percentage	0.00%
Duplicate Rows	0
Data Quality Score	1.0
Data Quality Status	Good

7.5 KPI and Business Insight Results

The pipeline computed the business KPIs shown in Table 7.3. Sales total $2,248,662.62 and profit $336,938.93. Category 1 leads on both sales and profit. region 2 is the strongest region and the natural priority for marketing, stock planning, and expansion. In the product interface every KPI carries a receipt revealing its calculation, and the complete result set is fingerprinted with SHA-256, so any figure can be re-verified against the raw data later.

Table 7.3: KPI Results

KPI	Result
Total Sales	$2,248,662.62
Total Profit	$336,938.93
Total Quantity Sold	69,546
Average Sales per Row	$64.25
Overall Profit Margin	14.98%
Best Region by Sales	Region 2
Best Category by Sales	Category 1
Best Category by Profit	Category 1

Best Store by Sales	Store 27

7.6 Visualization Results

Multiple visualizations were produced to support interpretation: sales and profit comparisons, customer type distribution, payment method usage, online versus offline, the monthly trend, and the discount-profit relationship. The compact executive report picks at most two charts automatically, each carrying a data-specific caption, axis labels, and a key takeaway line.

Table 7.4: Visualization Outputs

Figure	Title	Purpose
Figure 6.3	Sales by Category	Which category sells the most
Figure 6.4	Sales by Region	Which region performs best
Figure 6.5	Profit by Category	Which category earns the most profit
Figure 6.6	Monthly Sales Trend	How sales move over time
Figure 6.7	Customer Type Distribution	How customer segments split
Figure 6.8	Payment Method Distribution	How payment methods are used
Figure 6.9	Online vs Offline Sales	How the channels compare
Figure 6.10	Discount vs Profit Scatter	How discounting relates to profit

7.7 Report Generation Results

Report generation succeeded in both HTML and PDF. This capability matters because it turns raw analysis into a document students, business users, and decision makers can share. Reports observe a 2-page executive structure with deduplicated findings, business-readable prose free of internal jargon, KPI cards, and an Explainable AI section that always closes the document.

Table 7.5: Report Generation Results

Output	Result	Status
HTML Report	Generated successfully	Passed
PDF Report	Generated successfully within the 2-page budget	Passed
Charts in Report	Generated from the dataset with axis labels and takeaways	Passed
Business KPIs	Rendered from the dataset as KPI cards	Passed
Recommendations	Derived from the analysis (max 3, data-driven)	Passed

7.8 Natural Language Query and Agent Results

Agent behavior lives in the backend services. A DatasetAgent validates and understands each upload, while an AnalystAgent runs exploration, business metrics, chart planning, recommendations, and report preparation. An LLM provider layer supports OpenAI, Google Gemini, Anthropic Claude, and DeepAnalyze-8B when configured, and a deterministic Mock mode otherwise.

Table 7.6: Agent and LLM Integration Results

Check	Result
Dataset Agent	Implemented
Analyst Agent	Implemented
LLM Provider Layer	Implemented (OpenAI / Gemini / Claude / DeepAnalyze-8B)
DeepAnalyze Integration	Preferred agentic provider when configured; validated by automated tests
Mock/Fallback Mode	Available with no provider configured; analytics remain fully functional
Agent Output	Drives the dataset and analysis workflow
Security	API keys stay in the environment and never appear in reports

7.9 Prediction Results

Machine learning prediction is provided by the backend modeling service inside the analysis pipeline. The run completed with RandomForestClassifier selected and category as the target column, demonstrating that DataVerse AI goes beyond descriptive statistics into predictive analysis whenever a valid target exists.

Table 7.7: Prediction Result

Item	Result
Prediction Module	Implemented (Ridge regression / Random Forest with automatic task selection)
Model Used	RandomForestClassifier
Selected Target	category
Prediction Status	Complete
XAI Support	Implemented; SHAP with deterministic feature-importance fallback, plus counterfactuals

7.10 Supabase, Deployment, and Integration Results

Supabase is the production persistence layer: JWT authentication, PostgreSQL tables for sessions, messages, datasets, agent runs, and reports, and storage buckets for uploads and generated documents, with a local session cache speeding up repeated analysis. The application runs live: the frontend is hosted on Vercel and the FastAPI backend is reached through an ngrok secure tunnel on a reserved static domain. The complete journey, sign-up, upload, analysis, verification, and report download, was exercised in a browser against the deployed stack. Backend and frontend Dockerfiles provide a containerization path through Docker.

Table 7.8: Integration Results

Area	Result	Status
Frontend Upload Flow	Implemented	Passed
Backend Upload Endpoint	Implemented	Passed
CSV/Excel Parser	Implemented	Passed
Retail Domain Guard	Implemented; non-retail uploads refused before persistence	Passed
Report Download	Implemented	Passed
Supabase (Auth / DB / Storage)	Connected and required in production	Passed
Live Deployment	Vercel frontend + hosted FastAPI backend, verified end to end	Passed
CI (GitHub Actions)	Runs on every push and pull request	Passed
Frontend Build	Passed	Passed
Linting	Passed	Passed
Backend Tests	214 passed, 0 failed	Passed

7.11 Comparison with Manual Analysis

Set against manual Excel or Python work, the difference is effort. Manual analysis is capable but slow, skill-hungry, and repetitive. DataVerse AI automates profiling, KPIs, charts, recommendations, and reports.

Table 7.9: Manual Analysis vs DataVerse AI

Criteria	Manual Excel/Python Analysis	DataVerse AI
Technical Skill Required	Medium to high	Low
Dataset Profiling	Manual	Automatic
KPI Generation	Manual formulas/code	Automatic, with verifiable receipts
Chart Creation	Manual	Automatic
Natural Language Questions	Not available by default	Supported
Report Generation	Manual writing/export	Automatic HTML/PDF
Recommendations	Analyst-written	System-generated

Repeatability	Depends on user workflow	Repeatable upload-analysis workflow
Time Required	Higher	Lower

7.12 Discussion

The results show that DataVerse AI covers the core duties of an intelligent dataset analysis platform: loading, profiling, quality checking, KPI computation, charting, reporting, and prediction. In the evaluation data, region 2 delivered the highest sales, category 1 led both sales and profit, and store 27 was the top store. The online channel finished slightly ahead of offline on both sales and profit.

The discount analysis returned a weak negative correlation of -0.0739: discounts coincide mildly with lower profit, but the signal is too weak to steer decisions alone. Richer analysis mixing category, quantity, customer type, and seasonality would say more, and the product's root-cause investigation and verified what-if simulator exist precisely for that style of follow-up questioning.

7.13 Limitations

Some limitations remain. First, categorical fields arrive as numeric codes: region, city, category, customer type, and payment method are numbers, fine for analysis but hard to read. mapping tables would improve report readability. Second, the narration layer depends on external LLM availability and configuration. with no provider set, the application stays in deterministic Mock mode, which keeps every number correct but produces plainer prose. Third, serving the DeepAnalyze-8B agentic model at interactive speed needs GPU hosting (for instance vLLM on a 16 GB GPU), documented but not provisioned in the demo environment. Finally, the demo backend runs from a development machine through a secure tunnel. the documented cloud migration path removes that dependency.

7.14 Summary

This chapter reported the results of running DataVerse AI on the retail mart dataset. The application loaded and profiled 35,000 records, produced quality results, computed sales and profit KPIs, generated visualizations, created HTML/PDF reports, and demonstrated prediction with agent support. All 214 automated backend tests pass, Supabase is fully wired for authentication, persistence, and storage, and the entire flow was verified live on the deployed site,

including the retail-domain guard, which correctly refused a non-retail dataset. The evidence confirms the project's central claim: DataVerse AI lets users analyze retail data through an automated, verifiable, report-driven workflow.

References

[1] V. V. Meduri, A. Quamar, C. Lei, V. Efthymiou, and F. Ozcan, “BIREC: Guided Data Analysis for Conversational Business Intelligence,” arXiv, arXiv:2105.00467, May 2021. Accessed: Dec. 15, 2025. [Online]. Available: http://arxiv.org/abs/2105.00467

[2] J. Lian et al., “ChatBI: Towards Natural Language to Complex Business Intelligence SQL,” arXiv, arXiv:2405.00527, May 2024. Accessed: Dec. 15, 2025. [Online]. Available: http://arxiv.org/abs/2405.00527

[3] E. Cambria, L. Malandri, F. Mercorio, N. Nobani, and A. Seveso, “XAI meets LLMs: A Survey of the Relation between Explainable AI and Large Language Models,” arXiv, arXiv:2407.15248, July 2024. Accessed: Dec. 15, 2025. [Online]. Available: http://arxiv.org/abs/2407.15248

[4] “Survey on Explainable AI: From Approaches, Limitations and Applications Aspects | Human-Centric Intelligent Systems.” Accessed: Dec. 15, 2025. [Online]. Available: https://link.springer.com/article/10.1007/s44230-023-00038-y

[5] L. Shi, Z. Tang, N. Zhang, X. Zhang, and Z. Yang, “A Survey on Employing Large Language Models for Text-to-SQL Tasks,” ACM Comput. Surv., vol. 58, no. 2, pp. 1-37, Jan. 2026, doi: 10.1145/3737873.

[6] J. Schneider, “Explainable Generative AI (GenXAI): a survey, conceptualization, and research agenda,” Artif. Intell. Rev., vol. 57, no. 11, p. 289, Sept. 2024, doi: 10.1007/s10462-024-10916-x.

[7] P. Mavrepis, G. Makridis, G. Fatouros, V. Koukos, M. M. Separdani, and D. Kyriazis, “XAI for All: Can Large Language Models Simplify Explainable AI?,” arXiv, arXiv:2401.13110, Jan. 2024. Accessed: Dec. 15, 2025. [Online]. Available: http://arxiv.org/abs/2401.13110

[8] A. Plaat, M. van Duijn, N. van Stein, M. Preuss, P. van der Putten, and K. J. Batenburg, “Agentic Large Language Models, a survey,” arXiv, arXiv:2503.23037, Nov. 2025. Accessed: Dec. 15, 2025. [Online]. Available: http://arxiv.org/abs/2503.23037

[9] “A Research Landscape of Agentic AI and Large Language Models: Applications, Challenges and Future Directions.” Accessed: Dec. 15, 2025. [Online]. Available: https://www.mdpi.com/1999-4893/18/8/499

[10] RUC DataLab, “DeepAnalyze: Agentic Large Language Models for Autonomous Data Science,” GitHub repository. Accessed: July 12, 2026. [Online]. Available: https://github.com/ruc-datalab/DeepAnalyze